Numerical Methods in the n-Body Problem

Jake Wheldon - The University of Edinburgh

June & August 2025

1 Abstract

The n-body problem under Newtonian gravity was studied through numerical methods with the aim of locating and analyzing choreographic solutions. The equations of motion were formulated as coupled first-order ordinary differential equations, and trajectories were represented in Fourier space using discrete Fourier transforms. Numerical integration was carried out with Runge–Kutta, Euler–Cromer, Leapfrog, and Yoshida methods; the fourth-order Yoshida integrator was found to be the most effective, providing long-term stability with bounded oscillatory energy error. Suitability of integrators was assessed by observing conservation of energy, linear momentum, and angular momentum, while balancing numerical accuracy against computational cost. A minimization routine was developed in Fourier space using the principle of least action, with symmetry constraints imposed on Fourier coefficients to restrict the functional space. Classical results such as the figure-eight choreography were used to benchmark the method, highlighting both the effectiveness of symplectic schemes and the challenges of enforcing symmetry without introducing divergence.

2 Acknowledgments

I would like to thank my supervisor Dr Maximilian Ruffert, whose guidance and feedback made this project both navigable and enlightening.

Contents

1 Abstract **1**

2 Acknowledgments **2**

3 Introduction **4**

3.1 Overview 4

3.2 The Physical System 4

4 Numerical Integration **4**

4.1 Non-Symplectic Methods 5

4.2 Symplectic Methods 5

5 Numerical Minimization **13**

5.1 Approaching the Procedure 13

5.1.1 Defining Choreographies 13

5.1.2 Fourier Representation 15

5.2 Symmetric and Choreographic Groups 20

5.2.1 Notation 20

5.2.2 The Choreographic Group 20

5.2.3 Choreography Types 21

5.2.4 Fourier Coefficients of Planar Choreographies 22

5.3 Practical Implementation 23

5.3.1 Minimization in Practice 23

5.3.2 Developing the Minimizer 24

6 Further Topics and Conclusion **31**

A Source Code **32**

A.1 Integration 32

A.2 Minimization 43

3 Introduction

3.1 Overview

The n-body problem is a classical problem in mathematics and physics, which seeks to determine the motion of a system of celestial bodies that interact through gravitational forces.

The earliest solutions to the n-body problem come in 1734 and 1772, from Bernoulli [1] and Lagrange [2], respectively. These are physical orbits in the two- and three-body problem which consider the particles fixed at the vertices of a polygon rotating about its center at a fixed velocity. Around 200 years later, Moore discovered the figure-eight orbit of the three-body problem [3], later rediscovered in 2000 by Chenciner and Montogomery who proved its existence [4]. Many significant contributions have been made since then, through more complex manifestations of the symmetry-driven approach used by Lagrange and Bernoulli. Most notably, Simó alone has published over 300 solutions discovered using numerical methods [5], and existence proofs have been published for specific choreographies.

The main focus of this project is to investigate and document the numerical methods used in the process of choreography finding. We will investigate the minimization and integration techniques, with the aim of reproducing or finding choreographic solutions to the n-body problem.

3.2 The Physical System

We will be working with a Newtonian gravitational potential unless specified otherwise, and we denote the position of the ith body of unit mass as $\mathbf{x}_i(t)$. Using Newtonian gravity, the force on the body is given by

$$\mathbf{F}_i = \sum_{j \neq i} \frac{\mathbf{r}_{ij}}{||\mathbf{r}_{ij}||^3}$$

where $\mathbf{r}_{ij} = \mathbf{x}_j - \mathbf{x}_i$.

For our system, we can therefore give $2n$ equations of motion as first-order differential equations

$$\begin{cases} \dot{\mathbf{x}}_i(t) = \mathbf{v}_i(t) \\ \dot{\mathbf{v}}_i(t) = \sum_{j \neq i} \frac{\mathbf{r}_{ij}}{||\mathbf{r}_{ij}||^3} \end{cases}$$

with $\mathbf{v}_i$ representing the velocity of the ith particle.

4 Numerical Integration

Before we generate a set of initial conditions for the n-body choreography, we need a method to integrate the system to generate the full closed curve which will be used to provide an initial curve to check our minimizer as we develop it. There are many methods that have been developed for numerical integration. Using a known orbit, we can investigate the most

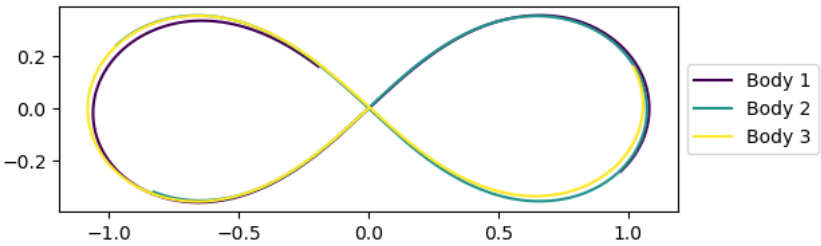


Figure 1: Runge-Kutta method with 5,000 time steps, dt = 0.01

suitable method by observing the behavior of conserved physical quantities over the course of integration. We expect these quantities to also be conserved throughout the integration process if we are to accurately replicate a physical system. Observing the conservation of energy, linear momentum, and angular momentum in the integration of the figure-8 orbit, we can compare the performance of symplectic and non-symplectic methods of varying orders.

The figure-8 orbit in this section will be generated with initial conditions

$$\begin{aligned} x_1 &= (0.97000436, -0.24308753), & v_1 &= (0.4662036850, 0.4323657300), \\ x_2 &= (0.97000436, -0.24308753), & v_2 &= (0.4662036850, 0.4323657300), \\ x_3 &= (0.00000000, 0.00000000), & v_3 &= (-0.93240737, -0.86473146). \end{aligned}$$

taken Simó [5].

4.1 Non-Symplectic Methods

We first consider the Runge-Kutta Method, as a convenient and powerful integrator with in-built Python programs. Using the *scipy.integrate.solve_ivp* program, we integrate the equations of motion with the figure-8 initial conditions to compare performance with other methods.

We see that whilst the orbit in Figure 1 doesn't necessarily diverge, there is some visual distortion in the form of rotation of the orbit, even over a short period.

The distortion of the orbit is confirmed when looking at the errors of the conserved quantities in Figure 2. We see that there is an increase in the energy and angular momentum error over the course of the orbit. Linear momentum has little to no error, remaining around machine precision.

Over 100,000 steps, the effect of this distortion is immediately noticeable in Figure 3 when observing the orbit alone. Considering the errors in conserved quantity shown in Figure 4, we see a noticeable energy drift and a significant error in the angular momentum.

Decreasing the step size to dt = 0.001, we still observe visual distortion of the figure-8 orbit in Figure 5. It is clear that the Runge-Kutta method is simply not suitable for integrating physical systems.

4.2 Symplectic Methods

Symplectic methods are commonly favorable when numerically integrating physical systems; the intuition and evidence for this will be explained over the course of this section. The intuition and

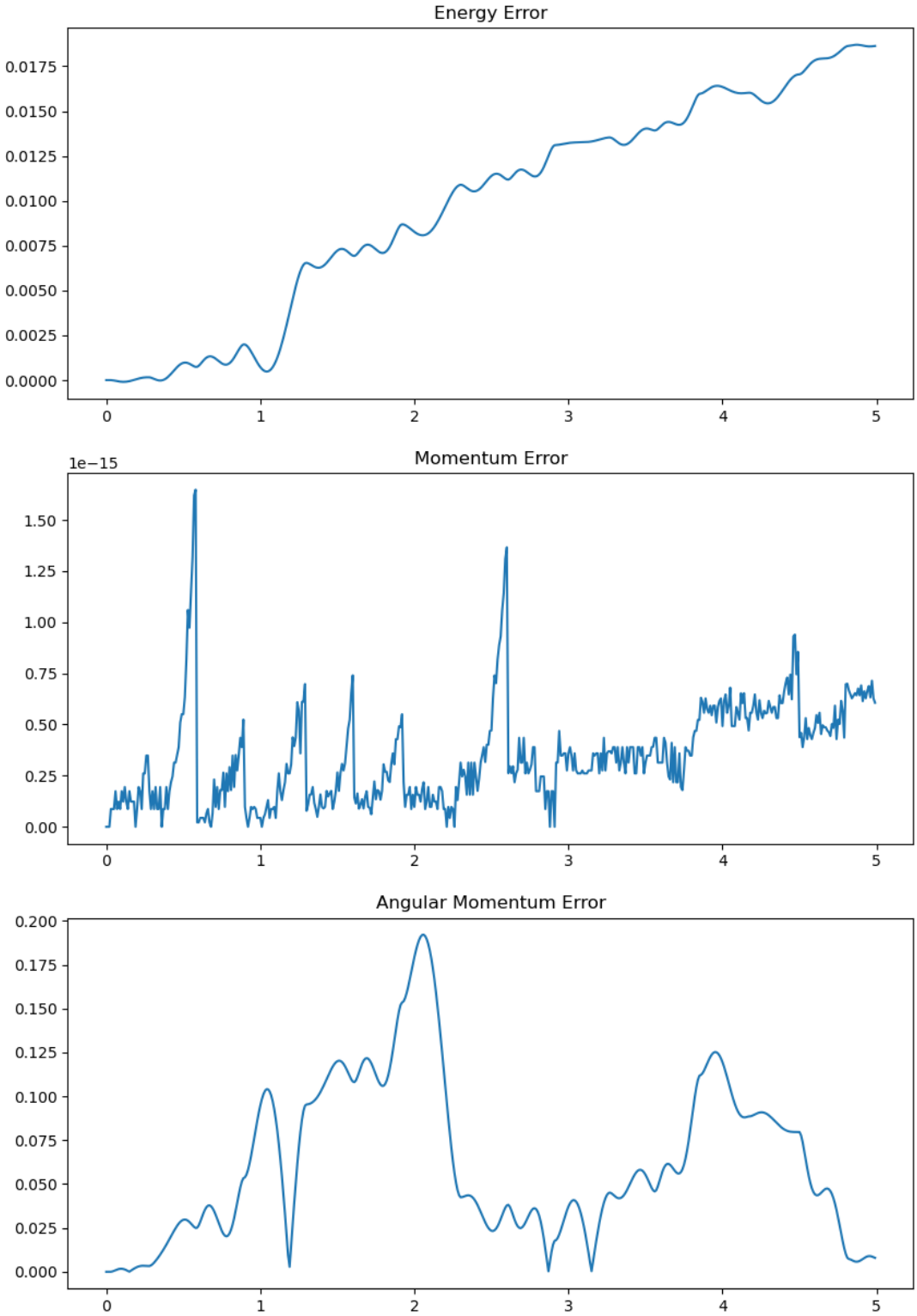


Figure 2: Errors for 5,000 time steps, dt = 0.01 with Runge-Kutta method.

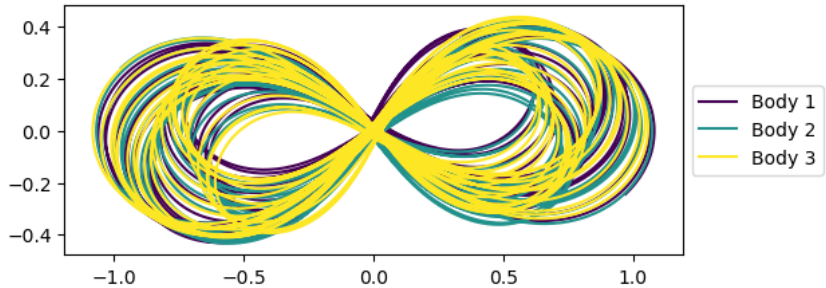


Figure 3: Runge-Kutta method with 100,000 time steps, dt = 0.01

formulations of more complex components are taken from Hairer, Wanner and Lubich [6] and Yoshida [7]. Firstly, we consider the simplest symplectic method, Euler-Cromer. The process for the ith body moving in three dimensions over a single time-step is laid out below

$$\begin{aligned}
\mathbf{a}_i^{(k)} &= A\big(\mathbf{x}_i^{(k)}\big) \\
\mathbf{v}_i^{(k+1)} &= \mathbf{v}_i^{(k)} + \mathbf{a}_i^{(k)}\Delta t \\
\mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+1)}\Delta t
\end{aligned}$$

where $A(\mathbf{x}_i\,;\,\{\mathbf{x}_j\}) = \sum_{i\neq j} G\frac{m_j(\mathbf{x}_j - \mathbf{x}_i)}{||\mathbf{x}_j - \mathbf{x}_i||^3}$.

We see it is a semi-implicit and so replicates the time reversibility of physical systems which is the symmetry from which energy conservation is derived. This provides some intuition as to why symplectic integrators do not have an energy drift. This is apparent when viewing the numerical integration of our equations of motion with the figure-8 initial conditions.

We see that our orbit in Figure 6 is produced with little distortion, and the linear and angular momentum errors are around machine precision suggesting the conservation of the two physical quantities, shown in Figure 7. We do see that the energy error is relatively high, but displaying an oscillatory behavior rather than a continuous upward drift.

Over a longer time period of 100,000 steps, we see in Figures 8 and 9 that whilst the energy error oscillates, it does not display any upward drift, instead showing a self-correcting behavior. The conservation of the physical quantities is also evidenced by the low angular momentum error, suggesting that this integration method does not cause a rotational distortion around the center of mass like the Runge-Kutta method did.

We now consider higher-order symplectic integrators. Firstly, the Leapfrog method. The process for the ith body moving in three dimensions in a single time-step is taken from [6] and laid out below

$$\begin{aligned}
\mathbf{a}_i^{(k)} &= A\big(\mathbf{x}_i^{(k)}\big) \\
\mathbf{v}_i^{(k+\frac{1}{2})} &= \mathbf{v}_i^{(k)} + \frac{1}{2}\mathbf{a}_i^{(k)}\Delta t \\
\mathbf{x}_i^{(k+1)} &= \mathbf{x}_i^{(k)} + \mathbf{v}_i^{(k+\frac{1}{2})}\Delta t \\
\mathbf{a}_i^{(k+1)} &= A\big(\mathbf{x}_i^{(k+1)}\big) \\
\mathbf{v}_i^{(k+1)} &= \mathbf{v}_i^{(k+\frac{1}{2})} + \frac{1}{2}\mathbf{a}_i^{(k+1)}\Delta t
\end{aligned}$$

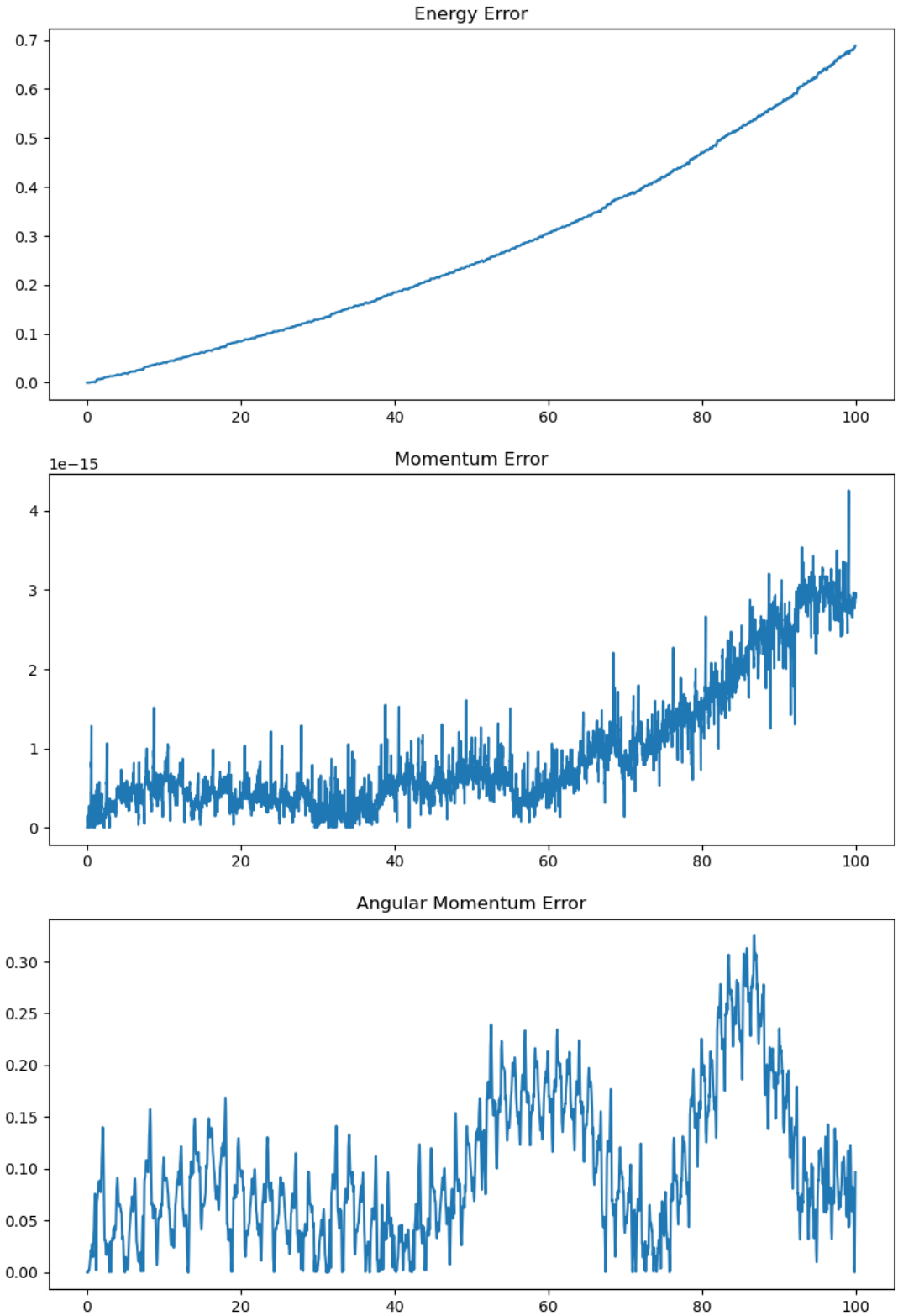


Figure 4: Errors for 100,000 time steps, dt = 0.01 with Runge-Kutta method.

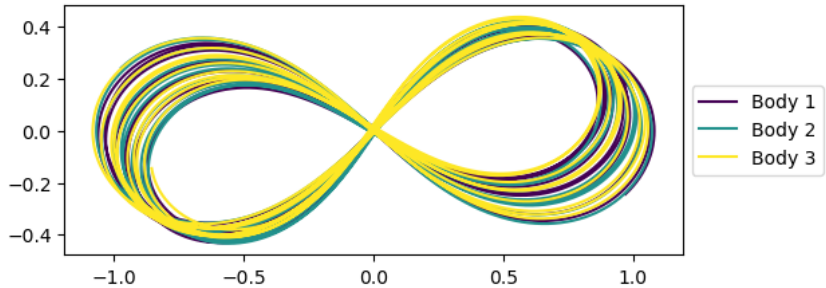


Figure 5: Runge-Kutta method with 50,000 time steps, dt = 0.001

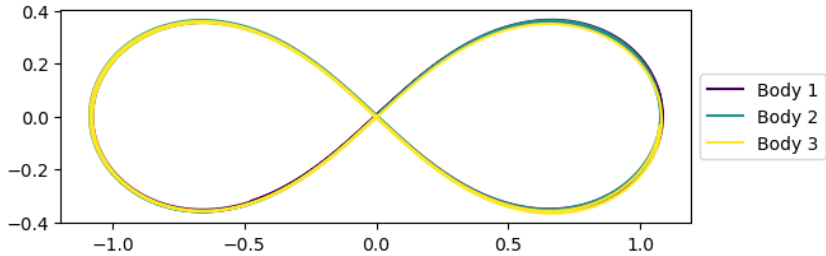


Figure 6: Euler-Cromer Method with 5,000 time steps, dt = 0.01

where $A(\mathbf{x}_i\,;\,\{\mathbf{x}_j\}) = \sum_{i\neq j} G\frac{m_j(\mathbf{x}_j-\mathbf{x}_i)}{||\mathbf{x}_j-\mathbf{x}_i||^3}$.

We see in Figure 11 that over a long time period of 100,000 steps, the amplitude of the energy error is smaller, and the variation in the peaks of the error curve is much less noticeable. Additionally, the linear and angular momentum errors remain near machine precision.

Finally, we look at the Yoshida method, a fourth-order integrator, with formulation taken from [7], given as follows

$$\begin{aligned}
\mathbf{x}_i^{(k+\frac{r}{4})} &= \mathbf{x}_i^{(k+\frac{r-1}{4})} + c_j\mathbf{v}_i^{(k+\frac{r-1}{4})}\Delta t,\\
\mathbf{a}_i^{(k+\frac{r}{4})} &= A\big(\mathbf{x}_i^{(k+\frac{r}{4})}\big),\\
\mathbf{v}_i^{(k+\frac{r}{4})} &= \mathbf{v}_i^{(k)} + d_j\mathbf{a}_i^{(k+\frac{r}{4})}\Delta t,
\end{aligned}$$

for $r = 1, 2, 3, 4$, where $A(\mathbf{x}_i\,;\,\{\mathbf{x}_j\}) = \sum_{i\neq j} G\frac{m_j(\mathbf{x}_j-\mathbf{x}_i)}{||\mathbf{x}_j-\mathbf{x}_i||^3}$ and

$$\begin{aligned}
c_1 &= \frac{w_1}{2}, & c_2 &= \frac{w_0+w_1}{2}, & c_3 &= \frac{w_0+w_1}{2}, & c_4 &= \frac{w_1}{2},\\
d_1 &= w_1, & d_2 &= w_0, & d_3 &= w_1, & d_4 &= 0,\\
& & w_0 &= \frac{-\sqrt[3]{2}}{2-\sqrt[3]{2}}, & w_1 &= \frac{1}{2-\sqrt[3]{2}}.
\end{aligned}$$

This method is made up of four leapfrog steps, and uses a "Drift-Kick-Drift" construction, advancing the positions, updating the accelerations, then advancing the velocities.

Observing the orbit below and the errors in Figure 13, we see linear and angular momentum errors around machine precision, and a small oscillating energy error with no observable upward drift over the long time period.

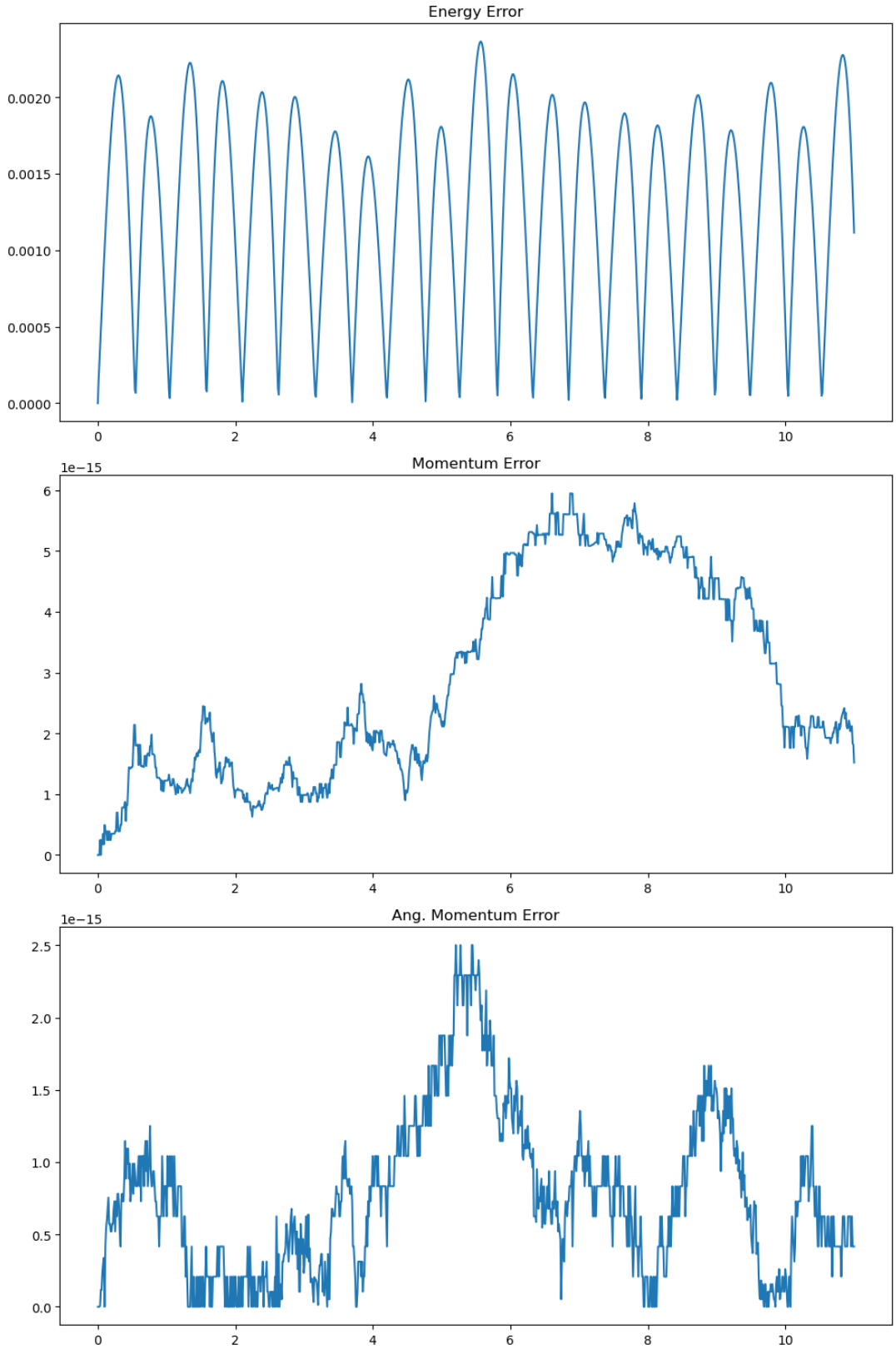


Figure 7: Errors for 5,000 time steps, dt = 0.01 with Euler-Cromer method.

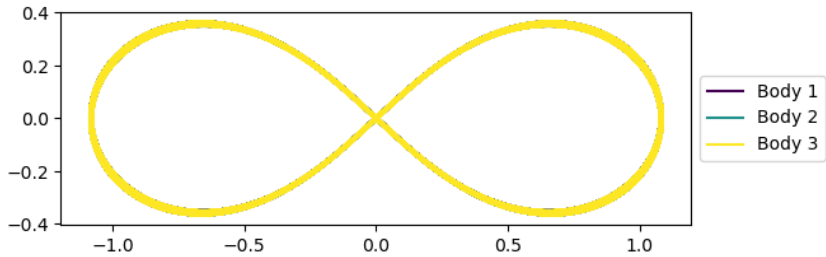


Figure 8: Euler-Cromer Method with 100,000 time steps, dt = 0.01

The Yoshida method will be the preferred method for integrating the initial conditions provided by the choreography minimizer. We provide some cases from the two-body problem to ensure it is working as expected

Considering the energy error associated with the perturbed circular motion in Figure 15, we see a pattern emerge that gives insight into the construction of symplectic integrators, built on more widely in [6].

We see that the energy error is largest at aphelion (when the bodies are furthest apart) and lowest at perihelion (when the bodies are closest together). Since the orbits are oscillatory, the energy error is also oscillatory, which explains the periodic energy error curve. This is a hallmark of symplectic integrators: rather than conserving the exact Hamiltonian of the system, they conserve a modified Hamiltonian that differs slightly from the true one by higher-order terms in the step size. The discrepancy between the two Hamiltonians is greatest when the bodies are farthest apart, as the modified dynamics deviate most strongly from the true gravitational potential in this regime. Consequently, the numerical solution exhibits bounded, oscillatory energy fluctuations rather than a secular drift, which is why symplectic methods remain stable and physically faithful over long time integrations.

A limitation of the Yoshida method is the computational power required to perform the integration. To mitigate this issue, we implement a variable time-step. Intuitively, we require a greater time resolution at points when velocities are higher, as the system is evolving most here. This suggests that at closest approach, we require a smaller time-step to model the system accurately. We use the following rule for our variable time-step

$$\Delta t_{\text{var}} = \eta \min_{i \neq j} \left(\frac{|x_i - x_j|}{|v_i - v_j|} \right)$$

where $\eta \in (0.01, 1)$ is scaling factor chosen depending on the required time resolution of the system. For many of our integrations it is set at 0.1.

The implementation of a variable time-step is a trade-off with the symplecticity of our integrator, as there is no guarantee that the system is now exactly time reversible. Consider the positions and velocities of all particles in the system in time-steps $k-1$, k, and $k+1$. $\Delta t_{k-1\to k}$ is dependent on the largest relative distance and velocity between two bodies in our system at the $(k-1)^{th}$ time-step. Moving back in time, $\Delta t_{k+1\to k}$ depends on the largest relative distance and velocity between two bodies in our system at the $(k+1)^{th}$ time-step. Our calculations for $\mathbf{x}^{(k)}$ and $\mathbf{v}^{(k)}$ rely directly on $\Delta t_{k-1\to k}$ or $\Delta t_{k+1\to k}$ depending on the direction we move in time. Hence, moving forward in time may produce a different $\mathbf{x}^{(k)}$ and $\mathbf{v}^{(k)}$ to moving backwards in time.

We observe the effect of the variable time-step implemented into our Yoshida integrator in Figure 16.

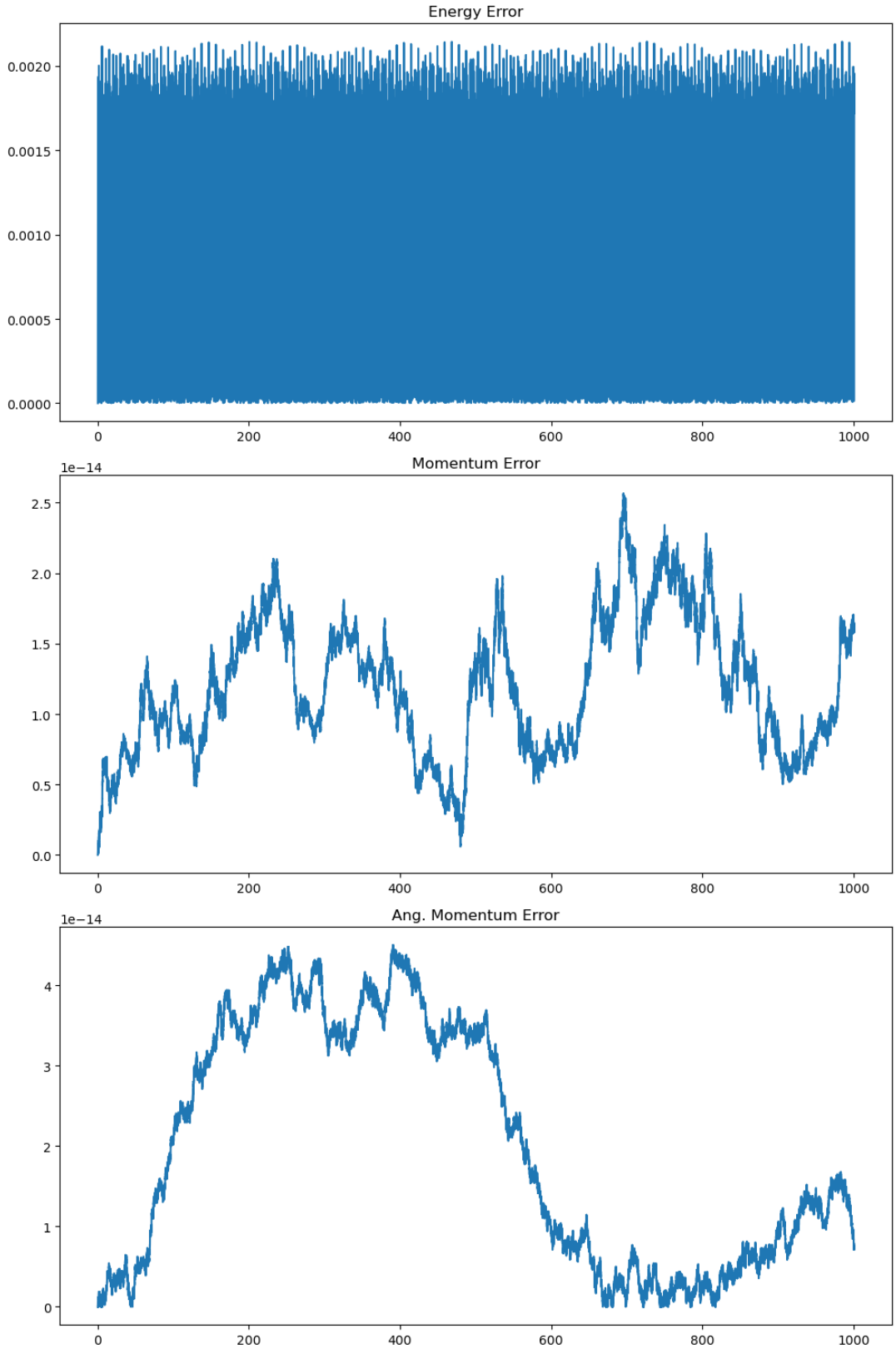


Figure 9: Errors for 100,000 time steps, dt = 0.01 with Euler-Cromer method.

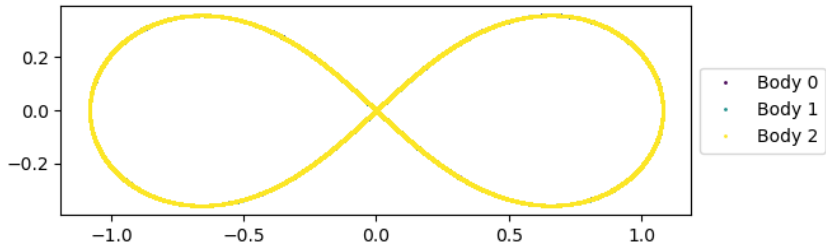


Figure 10: Leapfrog Method with 100,000 time steps, dt = 0.01

We see that there is a consistent upward drift of the energy error when we set $\eta = 0.3$ which implying the method is not symplectic as energy is not conserved. However, reducing η visibly reduces this upwards energy drift as well as the magnitude of the energy error. Linear and angular momentum errors remain around machine precision. With energy errors sufficiently small (0.0004%) we consider the Yoshida method with a variable time-step suitable for our purposes of integration over long periods when $\eta = 0.1$.

5 Numerical Minimization

Now that we have a way to integrate the equations of motion of the n-body problem from initial conditions, we look to find choreographies of the system. We aim to construct a minimizer that takes an "initial guess" trajectory and minimizes this within the functional space to a choreography of enforced symmetry.

5.1 Approaching the Procedure

5.1.1 Defining Choreographies

We formulate this problem mathematically by first defining a choreography. We adopt the following definition

Definition. A choreography is a system of n identical bodies where the position of the ith body at time t is given by a T-periodic function $f_i(t)$ with the property that $f_{i+1}(t) = f_i(t + \frac{T}{n})$ for all t, where the index is taken modulo n.

The principle of least action is central to finding choreographies numerically. We introduce the idea of the Hamiltonian action, denoted $\mathcal{S}$, which is a functional defined by the integral of the Lagrangian $\mathcal{L}$ along a trajectory between two instants in time

$$\mathcal{S}[\mathbf{x}(t)] = \int_{t_0}^{t_1} \mathcal{L}(\mathbf{x}(t), \dot{\mathbf{x}}(t))\, \mathrm{dt}$$

where we have $\mathbf{x}(t)$ as the trajectory of the particle. Hamilton's principle of least action states

Theorem. The equations of motion of a mechanical system coincides with the extremals of the action functional with Lagrangian

$$\mathcal{L}(\mathbf{x}, \dot{\mathbf{x}}, t) = T - U$$

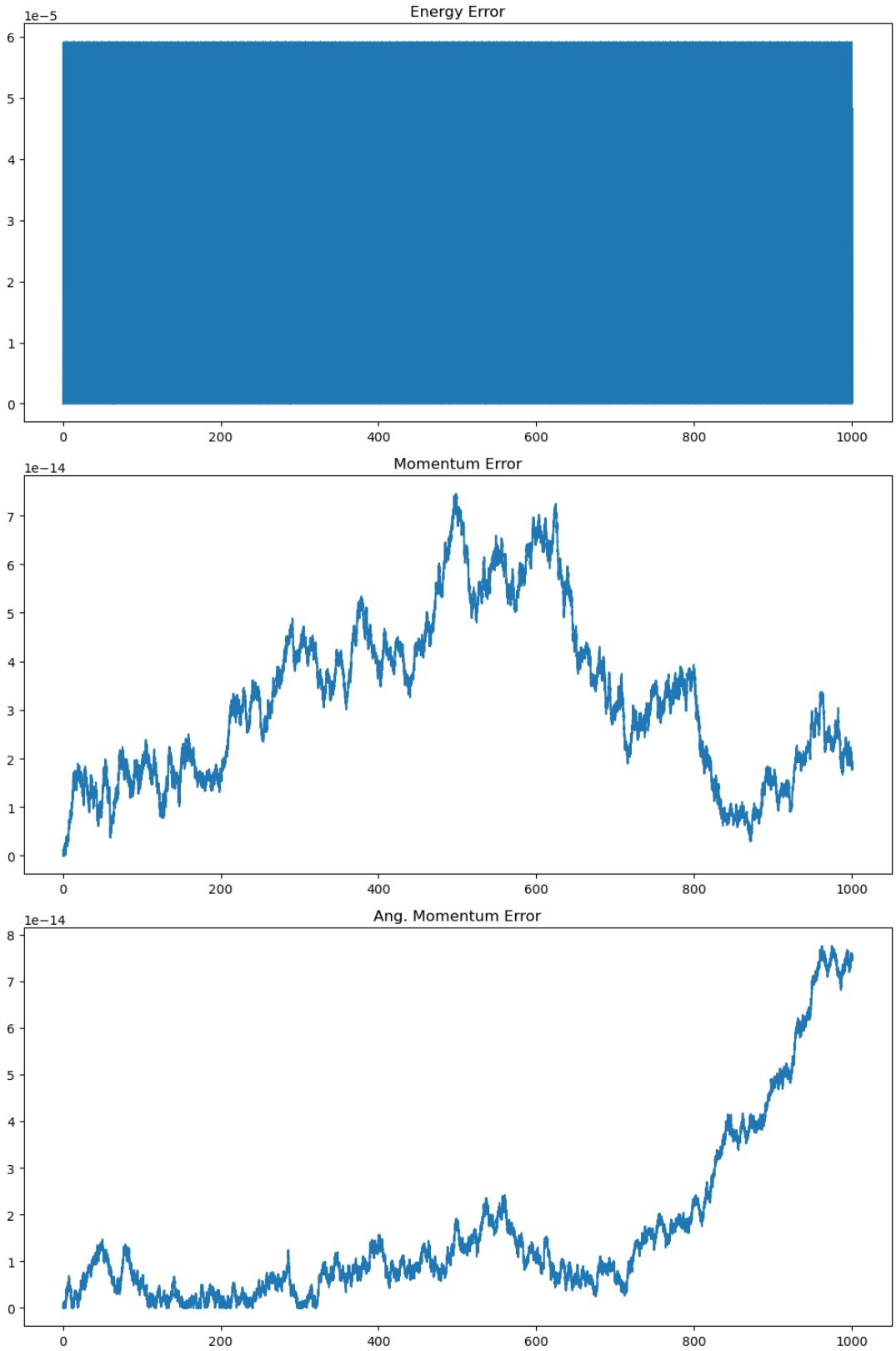


Figure 11: Errors for 100,000 time steps, dt = 0.01 with Leapfrog method.

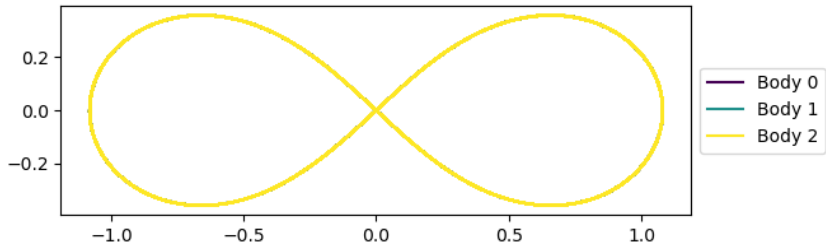


Figure 12: Yoshida method with 100,000 time steps, dt = 0.01

where T is the kinetic energy and U the potential energy of the system

Choreographies are physical orbits of our system, and so by the principle of least action they must minimize the Hamiltonian action of our system. We can explicitly define the values T and U for our system.

$$T = \sum_{i=1}^{n} \frac{1}{2} ||\dot{\mathbf{x}}_i(t)||^2$$
$$U = \sum_{i<j} \frac{1}{||\mathbf{x}_j(t) - \mathbf{x}_i(t)||}$$

So, the choreographies must minimize

$$\mathcal{S}[\mathbf{x}(t)] = \int_0^1 \left[\sum_{i=1}^{n} \frac{1}{2} ||\dot{\mathbf{x}}_i(t)||^2 + \sum_{i<j} \frac{1}{||\mathbf{x}_j(t) - \mathbf{x}_i(t)||} \right] \mathrm{dt}$$

Notice that we have integrated from $t_0 = 0$ to $t_1 = 1$. We have given our choreography an arbitrary period of $T = 1$, which is to be assumed moving forward.

5.1.2 Fourier Representation

The periodic nature of choreographies means that they can be accurately represented as Fourier series. We can represent them in complex form

$$z(t) = \sum_{k \in \mathbb{Z}} \zeta_k \exp(i\omega_k t)$$

where $\omega_k = \frac{2\pi k}{T}$. Equivalently, using $\exp(i\omega_k kt) = \cos(\omega_k t) + i\sin(\omega_k t)$ and $\zeta_k = a_k + ib_k$, we consider the motion in each of the x and y directions separately

$$x(t) = \sum_{k \in \mathbb{Z}} a_k \cos(\omega_k t) - b_k \sin(\omega_k t)$$
$$y(t) = \sum_{k \in \mathbb{Z}} a_k \sin(\omega_k t) + b_k \cos(\omega_k t)$$

By term-wise differentiation, we can represent the velocity in the x and y directions with the same coefficients

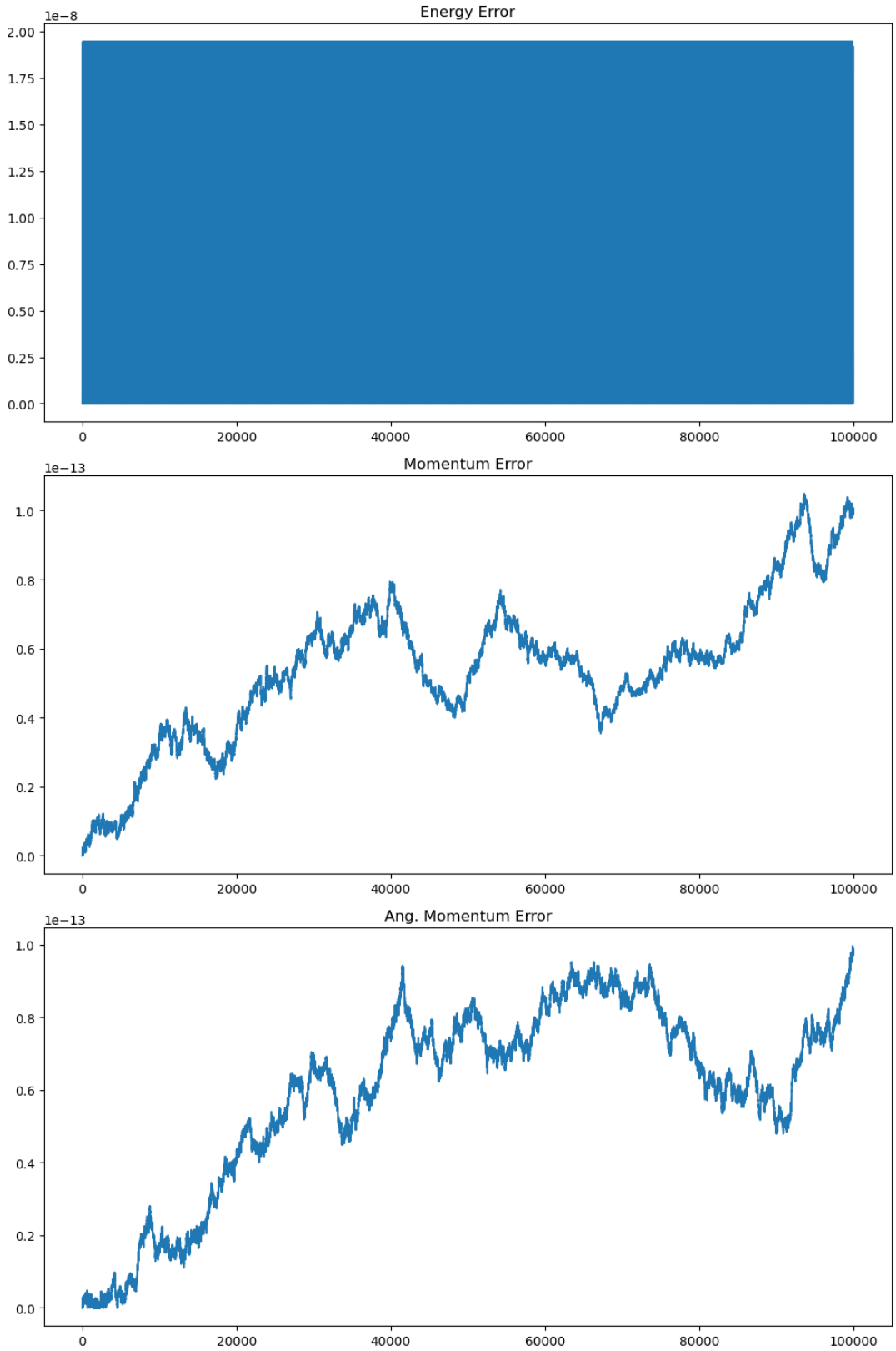
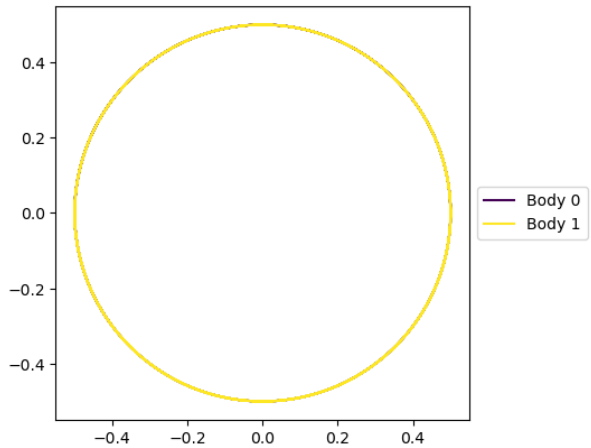
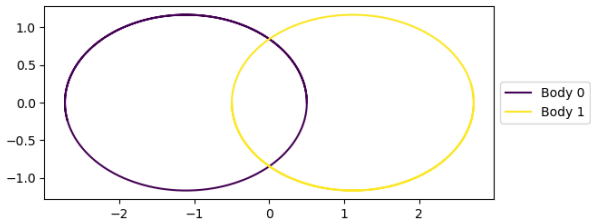


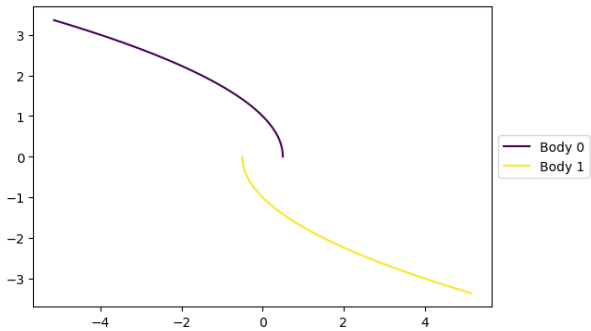
Figure 13: Errors for 100,000 time steps, dt = 0.01 with Yoshida method.



(a) Circular Motiom

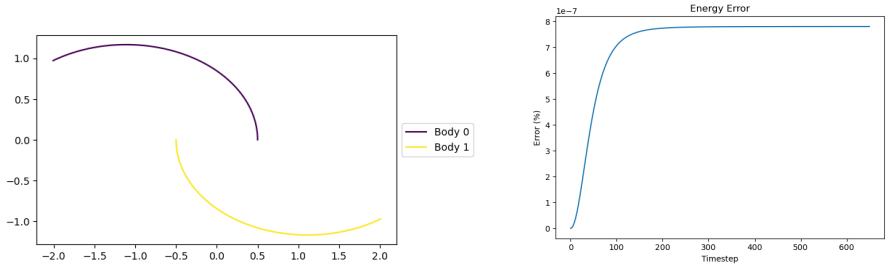


(b) Perturbed Circular Motion

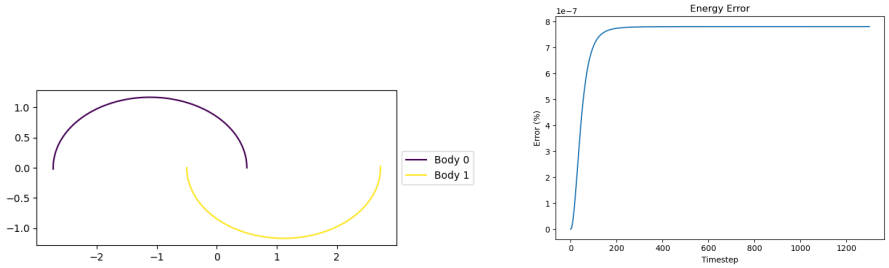


(c) Parabolic Motion

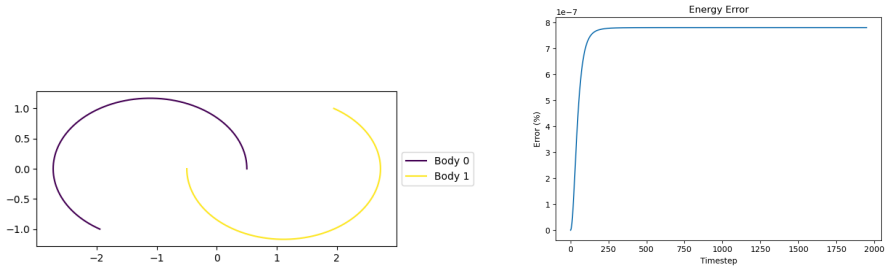
Figure 14: Two body orbit initial conditions with Yoshida integrator



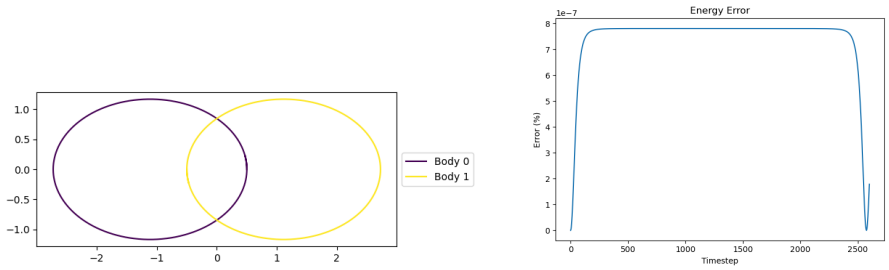
(a) 650 Steps



(b) 1300 Steps

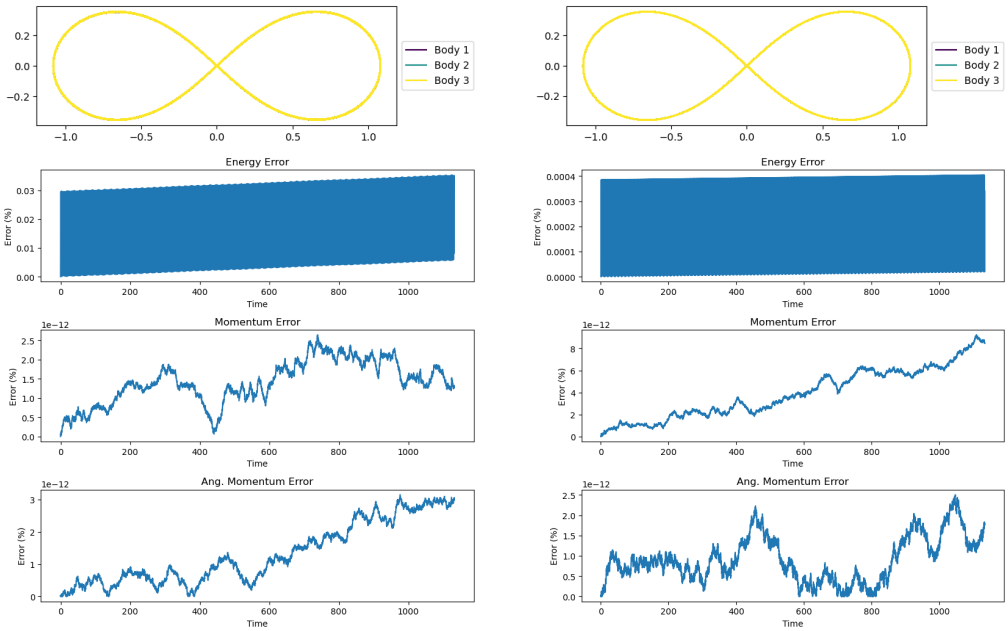


(c) 1950 Steps



(d) 2600 Steps

Figure 15: Evolution of perturbed circular motion over time



(a) $\eta = 0.3$ **(b)** $\eta = 0.1$

Figure 16: Comparison of Yoshida method with variable time-step for $\eta = 0.1$ and 0.3

$$v_x(t) = \sum_{k\in\mathbb{Z}} -a_k\omega_k \sin(\omega_k t) - b_k\omega_k \cos(\omega_k t)$$
$$v_y(t) = \sum_{k\in\mathbb{Z}} a_k\omega_k \cos(kt) - b_k\omega_k \sin(\omega_k t)$$

We will see that this is incredibly important in helping us develop a tool to minimize the action of an initial guess to a physical trajectory through numerical methods.

So far, we have considered the trajectories of our system as continuous. In our context, the reality is that their positions will be generated at discrete time steps by a numerical integrator. A useful way to consider these discretized curves in Fourier space is by using a Discrete Fourier Transform. Let $f(t)$ be the continuous signal of source data. The N samples are denoted $f[0], f[1], ..., f[N-1]$. Then the Discrete Fourier Transform of the signal is given

$$F_k = \sum_{k=0}^{N-1} f[k]e^{-2\pi i \frac{kn}{N}} \text{ for } k = 0, 1, ..., N-1$$

And so the continuous function can be approximated on $t \in [0, T)$ by

$$f(t) \approx \frac{1}{N} \sum_{k=0}^{N-1} F_k e^{2\pi i \frac{kt}{T}}.$$

This is an approximation, as there are finitely many terms in the Fourier series due to physical limitations. As $N \to \infty$, the DFT approaches the continuous Fourier Series, but for our purposes $N > 100$ is generally sufficient.

5.2 Symmetric and Choreographic Groups

Choreographies are symmetric in time by definition. They are periodic trajectories, with particles evenly spaced throughout the closed curve. Considering the shapes of known choreographies, we see that the figure-8 and equilateral triangle trajectories exhibit spacial symmetries too. We now consider how to impose these spatial symmetries mathematically, in a practical sense this will change the functional space that we perform our minimization routine on. Most importantly, the link between the symmetry of a choreography and its representative Fourier coefficients is profound and a necessary link to make in order to develop our choreography finder.

Throughout this section, we will see some justification for this approach through coercivity arguments, with the mathematical basis mostly drawn from the work of Montaldi and Steckles[8], and Ferrario and Terracini [9] as well as inspiration from [10] and [11].

5.2.1 Notation

We begin by noting the groups and conventions we will refer to in this section. We consider time as cyclical due to the periodic nature of the choreographies. To do so, time t is defined on the time circle $\mathbb{T} = \mathbb{R}/T\mathbb{Z}$, where T is the period of the choreography. We typically set $T = 1$, giving $\hat{\mathbb{T}} = \mathbb{R}/\mathbb{Z}$.

We work with the following groups

- Cyclic Group, $\mathbb{Z}_n$, the rotational symmetries of a regular n-gon.
- Dihedral Group, D_n, symmetries of a regular n-gon.
- Symmetric Group, S_n, the permutations of n objects.
- Orthogonal Groups, $O(2)$ and $\hat{S}^1$, reflections and rotations in space and on the time circle respectively, where $O(2) \cong \hat{S}^1$.

5.2.2 The Choreographic Group

We are looking for planar choreographies, so we identify the two dimensions of motion with the real and imaginary axis of the complex plane as it lends itself naturally to mathematical definitions of reflections and rotations. The positions of the n bodies are given as z_1, z_2, ..., $z_n \in \mathbb{C}$. All particles have unit mass, and so the center of mass of our system is given by

$$CoM = \frac{1}{n}\sum_i z_i$$

By Galilean relativity, we can fix this at the origin without loss of generality. Our configuration space is therefore

$$X_n := \left\{(z_1, z_2, ..., z_n) \in \mathbb{C}^n \,\middle|\, \sum_i z_i = 0, z_i \neq z_j, \forall i \neq j\right\}$$

This definition fixes the center of mass of the system at the origin and is collision-free.

Our aim is now to restrict the functional space to a subset consisting solely of choreographies with the desired spatial symmetries. We will later see how to practically enforce these symmetries through the Fourier coefficients of the loop, and hence how to ensure minimization occurs on the choreographic subset of the functional space.

We denote the loop space, the space of continuous loops $f : \hat{\mathbb{T}} \to X_n$ in our configuration space, by Λ. Three natural symmetries apply to this space

- Rotations and reflections via $O(2)$.
- Permutations of the identical particles via S_n.
- Translations and reflections in time via $\hat{S}^1$

The general symmetry acting on loop space Λ is therefore $G = O(2) \times S_n \times \hat{S}^1$. We say that a loop x_n has a symmetry $H < G$ if x_n is invariant under H. From the above, we take two important definitions.

Definition. The **choreographic generator** c_n is defined $c_n = (\mathbb{I}, \sigma_1, -\frac{1}{n}) \in G$ where $\mathbb{I}$ is the identity and $\sigma_1 = (1\,2 \ldots n) \in S_n$.

The generator enforces the defining characteristic of a choreography, that the $i+1^{th}$ ordered particle follows the same path as the ith particle, but with a time delay of $-1/n$ when we consider the unit time circle, hence $c_n^n = (\mathbb{I}, \mathrm{id.}, -1)$. Notice that this requires no spatial symmetry to be enforced, only that all particles are evenly spaced on the same path.

Definition. The **choreographic group** $\mathcal{C}_n$ is the cyclic group of order n generated by c_n.

This gives rise an alternative definition of a choreography from the perspective of our defined groups

Definition. A loop is a **choreography** if and only if it is an element of $\mathrm{Fix}(\mathcal{C}_n, \Lambda)$.

5.2.3 Choreography Types

We use the insight into the choreography group to break down and formalize the types of symmetry a choreography can have. To do this, consider $z_i(t)$ as the position of the ith particle at time t

- **Reflections** in the x-axis are denoted by $k \in O(2)$, or equivalently by complex conjugation, so $k : z_i(t) \to \overline{z}_i(t)$,
- Anticlockwise **rotations** are denoted $R_\theta \in O(2)$, so $R_\theta : z_i(t) \to e^{i\theta} z_i(t)$,
- $\sigma_k \in S_n$ denotes the unique **permutation** such that $\sigma_k^k = (1\,2 \ldots n)$, which acts on particle labels such that $\sigma_k : z_i(t) \to z_{\sigma_k(i)}(t)$,
- **Time translations** are denoted $\varphi \in \hat{S}^1$ so $\varphi : t \to t + \varphi$. Additionally, $\overline{0}$ denotes a reversal of time, so $\overline{0} : t \to -t$.

Subgroup	Generators	Isomorphic to
$C(n, p/l)$	$\{c_n,\ (R_{2\pi l/p},\ e,\ 1/p)\}$	$\mathbb{Z}_n \times \mathbb{Z}_p$
$D(n, p/l)$	$\{c_n,\ (R_{2\pi l/p},\ e,\ 1/p), (k,\ s_1,\ \overline{0})\}$	$(\mathbb{Z}_n \times \mathbb{Z}_p) \rtimes \mathbb{Z}_2$

Table 1: Subgroups of C_n for all $n > 2$

Subgroup	Generators	Isomorphic to
$C'(n, 2)$	$\{(k,\ \sigma_2,\ -1/2n)\}$	$\mathbb{Z}_{2n}$
$D'(n, 1)$	$\{(R_\pi,\ s_1,\ \overline{0})\}$	D_{2n}
$D'(n, 2)$	$\{(k,\ \sigma_2, -1/2n),\ (R_\pi,\ s_1,\ \overline{0})\}$	D_{4n}

Table 2: Subgroups of C_n for odd $n > 2$

Using this, we can list the possible groups for planar choreographies in Tables 1 and 2.

Here l and p are taken as co-prime integers, e is the identity permutation and s_1 is the permutation reversing the labeling of particles and σ_2 is a permutation of the indices of particles by two, e.g $(1\,3\,5\,2\,4)$, which is why n must be odd as no such permutation exists for even n. This notation comes from Schlafli symbols, in accordance with [8].

It is from Montaldi that we take the following, with the proof found in [8].

Theorem. The symmetry group of any planar choreography is either circular or is conjugate to one of the above subgroups depending on n being odd or even.

5.2.4 Fourier Coefficients of Planar Choreographies

Identifying the subgroups of the choreographic group allows us to classify any choreographies our minimizer may return in a meaningful way. It also provides insight into the Fourier coefficients of choreographies when we move them into Fourier space. From the above groups, we know which Fourier coefficients to perturb and the symmetries they should obey. For example, we want to look for choreographies in a center-of-mass frame, and so for all n bodies we require

$$\begin{aligned} 0 &= \frac{1}{n}\sum_{j=0}^{n-1} z_j(t) \\ &= \frac{1}{n}\sum_{j=0}^{n-1}\sum_{k\in\mathbb{Z}} \zeta_k \exp(i\omega_k(t + j/n)) \text{ as } z_j(t) = z_0(t + j/n) \\ &= \frac{1}{n}\sum_{k\in\mathbb{Z}} \left(\exp(i\omega_k t) \sum_{j=0}^{n-1} \zeta_k \exp(i\omega_k j/n) \right) \end{aligned}$$

which holds for all t. Hence, for each $k \in \mathbb{Z}$, $\zeta_k = 0$ and / or $\sum_{j=0}^{n-1} \zeta_k \exp(i\omega_k j/n) = 0$. Considering cases of n

- If $n|k$, then we have $\exp(i\omega_k j/n) = 1$ and so $\sum_{j=0}^{n-1} \zeta_k \exp(i\omega_k j/n) = n$
- If $n \nmid k$, then we have a geometric series $\sum_{j=0}^{n-1} \zeta_k \exp(i\omega_k j/n) = \frac{1-\exp(i2\pi k(n+1)/Tn)}{1-\exp(i2\pi k/Tn)} = 0$ as required.

Subgroup	Condition on Fourier coefficients
$C(n, p/l)$	$\zeta_r = 0 \ \forall r \neq l \mod p$
$D(n, p/l)$	$\zeta_r = 0 \ \forall r \neq l \mod p$, $\zeta \in \mathbb{R}$
$C'(n, 2)$	$\zeta_{-r} = (-1)^r \overline{\zeta}_r$
$D'(n, 1)$	$\zeta_{-r} = -\zeta_r$
$D'(n, 2)$	$\zeta_{-r} = -\zeta_r$ with $\zeta_r \in \mathbb{R}$ for even r and $\zeta_r \in i\mathbb{R}$ for odd r.

Table 3: Restrictions on Fourier coefficients for each subgroup of $\mathcal{C}_n$.

So, for the center-of-mass requirement to hold, we need $\zeta_k = 0$ for $k = 0 \mod n$. This restriction on the Fourier coefficients applies to all of the symmetry groups, and so it will be enforced regardless of which type of choreography we look for.

This kind of process can be carried out on all symmetry groups, the results described in the Table 3, taken from [10].

5.3 Practical Implementation

5.3.1 Minimization in Practice

In this next section, we consider how to enforce the minimization to certain functional spaces practically.

To successfully minimize an initial-guess trajectory to a choreography of the required symmetry, we need to find a loop of stationary action within the correct functional space using calculus of variations. Analytically this problem is hard to attack, with only the simplest cases having had any progress, for example the proof of the existence of the figure-8 orbit by Chenciner and Montgomery. We consider a numerical approach.

Let $\mathcal{S} : \Lambda \to \mathbb{R}$ be any action functional such that it has critical points corresponding to choreographies, that is the solutions are periodic and invariant under the action of $G = O(2) \times S_n \times \hat{S}^1$ as defined earlier. Considering the subgroup $H < G$. We then restrict $\mathcal{S}$ to $\mathrm{Fix}(H, \Lambda)$ to ensure we minimize within a choreographic space. Denote this $\mathcal{S}^H$. We know that if $\mathcal{S}^H$ is coercive:

$$||x|| \to \infty \implies ||f(x)|| \to \infty,$$

then it is guaranteed to have a minima on connected components of $\mathrm{Fix}(H, \Lambda)$ as this ensures continuity and that the minima is attained.

Ferrario and Terracini show that the restriction of the functional action to loops in the completion of X, denoted $\overline{X}_n \simeq \mathbb{C}^{n-1}$, with symmetry $H < G$ is coercive if and only if

$$\mathrm{Fix}(G, \overline{X}_n) = \{0\}.$$

The intuition behind this result is that the invariance of the action of certain loops under the symmetry of G allows the norm of the loops to grow arbitrarily large which the action is unchanged. Take a translational symmetry for example, if the action is fixed with relation to this symmetry we can take loops arbitrarily far away that have the same action, implying that there is a flat direction in our functional space which the minimizer could move along. Ensuring there are no nontrivial fixed action functionals under the symmetry means that the action grows in all directions and coercivity holds.

The importance of this result is that we do not have to worry about the divergence of our minimization in this space provided we enforce the symmetry conditions.

5.3.2 Developing the Minimizer

The functional minimizer we will be using is *scipy.optimize.minimize*, acting on the action functional within Fourier space. This allows us to perturb the Fourier coefficients of our loop in order to minimize its action.

We work in Fourier space as a way to restrict our action functional to $\mathrm{Fix}(H, \Lambda)$ and ensure coercivity, this is done by enforcing the desired symmetries of the resulting choreography through the conditions on the Fourier coefficients laid out in Table 3. This also has a benefit in increasing the efficiency of our minimizer as we have to perturb less parameters, and the velocity of the bodies along the trajectory can be found by term-wise differentiation of the trajectory in Fourier space, as discussed in Section 4.2.4.

As a result, we are looking for Fourier coefficients $\zeta_r \in \mathbb{C}$ such that

$$\frac{dS[z(t)]}{d\zeta_r} = 0 \ \forall r \in \mathbb{Z}.$$

Using our work up to now, we can develop an outline for finding choreographies of the n-body problem

- Generate an initial guess for a choreography roughly satisfying your required symmetries and calculate the action of the trajectory.
- Sample K points (x, y) from the initial path, and generate K Fourier coefficients $\{\zeta_1, ..., \zeta_K\}$using a Discrete Fourier Transform.
- Impose the symmetry conditions outlined in Table 3 on the initial Fourier coefficients.
- Minimize the action functional $S[\mathbf{x}]$ parametrized by the Fourier coefficients using numerical methods.
- Shift the minimized Fourier coefficients by $e^{\frac{-i\omega_k j}{N}}$ for $j = 0, 1, ..., n$. This is equivalent to a rotation in time by $-\frac{j}{n}$ for $i = 0, 1, ..., n$ to generate the Fourier coefficients of a trajectory for each body in the system, denoted $\{\zeta_0^{(i)}, ..., \zeta_K^{(i)}\}$ for $i = 0, 1, ..., n$.
- Differentiate the Fourier series of each of the trajectories to generate the Fourier coefficients of the velocities of each of the bodies, given $\{i\omega_0\zeta_0^{(i)}, ..., i\omega_K\zeta_K^{(i)}\}$ for $i = 0, 1, ..., n$.
- Transform all Fourier coefficients generated back into the configuration space, and calculate the action of the minimized trajectory. Confirm it is lower than the action of the initial guess for the choreography.
- Take the first entries $(\mathbf{x}_0^{(i)}, \mathbf{v}_0^{(i)})$ for $i = 0, 1, , ..., n$ as initial conditions.
- Integrate the initial conditions in the Yoshida integrator to confirm existence of the choreography.

We start with a sample of the figure-8 orbit. To consider the Fourier coefficients of the x and y paths, we use *numpy.fft.rfft* to perform the Discrete Fourier Transform on a sample of K points (x, y) sampled evenly in time taken from integrating the initial conditions

$$\begin{aligned} x_1 &= (0.00000000, 0.00000000), & v_1 &= (-0.93240737, -0.86473146), \\ x_2 &= (0.97000436, -0.24308753), & v_2 &= (0.4662036850, 0.4323657300), \\ x_3 &= (0.97000436, -0.24308753), & v_3 &= (0.4662036850, 0.4323657300). \end{aligned}$$

for the figure-8 over one full period of $T = 6.3259$. This provides two sets of Fourier coefficients for x and y respectively.

Initially, as we are working with the figure-8 orbit, to ensure minimization we enforce the symmetry conditions laid out for $D'(n, 2)$ in Table 3. These conditions are requirements on ζ_r, the coefficients of the trajectory in the complex plane, but we are working with x and y as separate Fourier series. We must slightly change our restrictions on the coefficients. Python returns a set of complex Fourier coefficients which I will denote $\{\alpha_k\}$ and $\{\beta_k\}$ for the x and y coefficients respectively. We have our restrictions in terms of $\zeta_k = a_k + ib_k$ where a_k and b_k are the coefficients of our x and y Fourier series, seen

$$\begin{aligned} x(t) &= \sum_{k\in\mathbb{Z}} a_k \cos(kt) - b_k \sin(kt) \\ y(t) &= \sum_{k\in\mathbb{Z}} a_k \sin(kt) + b_k \cos(kt) \end{aligned}$$

By relating the Fourier coefficients denoted above

$$\begin{aligned} a_k &= \frac{\alpha_k - \overline{\alpha}_k}{2}, \quad -b_k = \frac{\alpha_k + \overline{\alpha}_k}{2i} \\ b_k &= \frac{\beta_k - \overline{\beta}_k}{2}, \quad a_k = \frac{\beta_k + \overline{\beta}_k}{2i} \end{aligned}$$

The restrictions require that

$$\begin{aligned} \text{For even } k,\ \ \zeta_r \in \mathbb{R},\ \text{so } b_k = 0 &\implies \alpha_k \in \mathbb{R},\ \ \beta_k \in \mathbb{C} \\ \text{For odd } k,\ \ \zeta_r \in \mathbb{C},\ \text{so } a_k = 0 &\implies \alpha_k \in \mathbb{C},\ \ \beta_k \in \mathbb{R} \end{aligned}$$

We also require anti-symmetry $\zeta_{-r} = -\zeta_r$, which works out as $\alpha_{-k} = -\alpha_k$ and $\beta_{-k} = -\beta_k$.

Enforcing these constraints through program *Symmetry_D2dash*, we observe the effect on the figure-8 orbit in Figure 17.

We see no observable effect on the trajectory once restricted to this symmetric group as expected. Observing the effect on the Fourier coefficients themselves, we see something slightly different in Figure 18. The coefficients of the original curve in fact do not follow the restrictions that we imposed for the figure-8 trajectory. Due to time limitations, and the fact that Table 3 of coefficient restrictions is referenced from elsewhere, this is not something that I entirely understand yet.

When observing the complex Fourier coefficients of the $z(t)$ trajectory in Figure 19 generated by the Yoshida integrator directly, they follow the restrictions on ζ_r to $\approx 10^{-4}$. This is likely due to accumulating errors in the numerical integration, and so reinforces that the restrictions of $D'(n, 2)$ in Table 3 are correct. This would suggest that the conversion from $z(t)$ to $x(t), y(t)$ coefficient restrictions is therefore too strict. This is something to look into more closely in future.

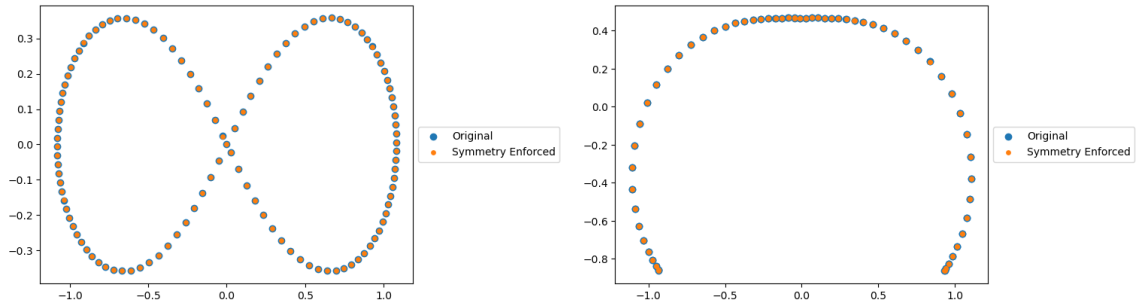
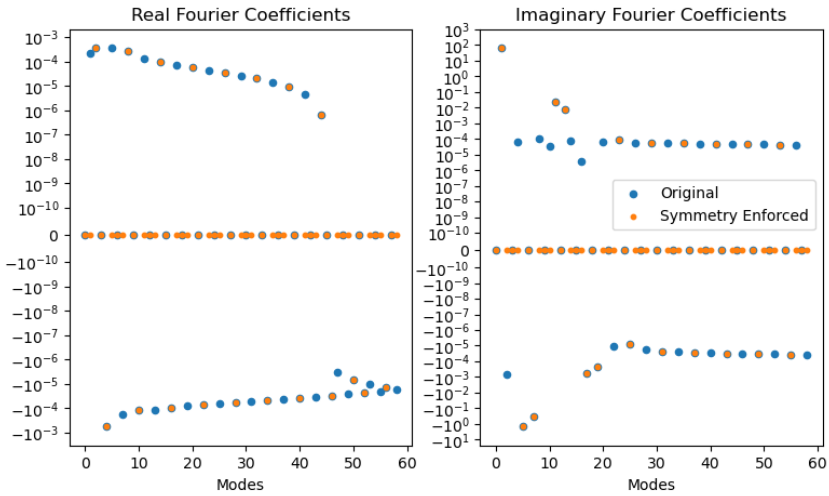
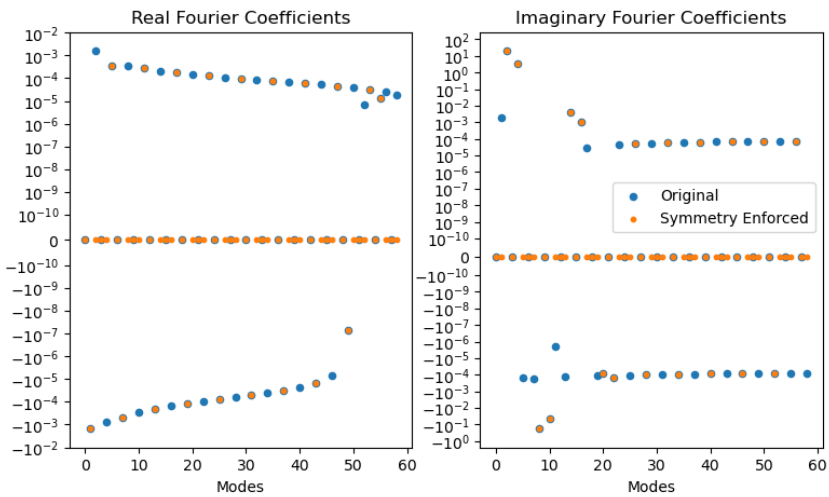


Figure 17: Figure-8 trajectory and velocity, 132 samples, with D'(n, 2) restrictions on Fourier coefficients.



(a) $x(t)$ Fourier coefficients



(b) $y(t)$ Fourier coefficients

Figure 18: Fourier coefficients of the figure-8 before and after applying symmetry restrictions.

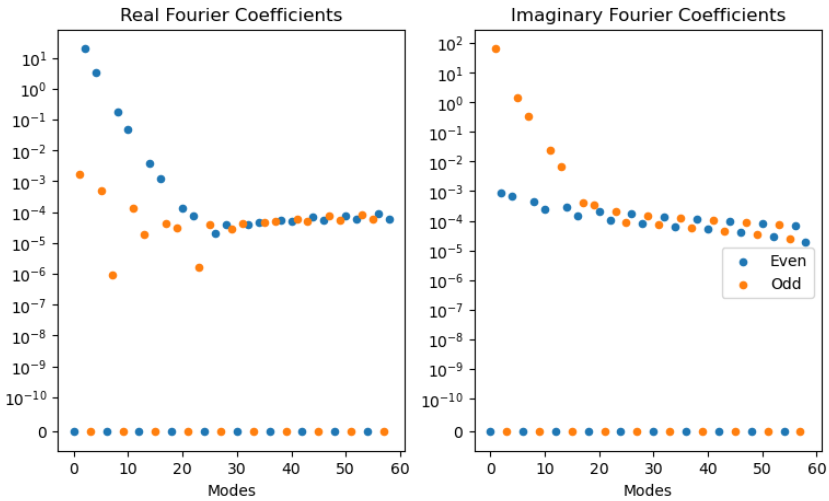


Figure 19: Absolute values of z(t) Fourier coefficients

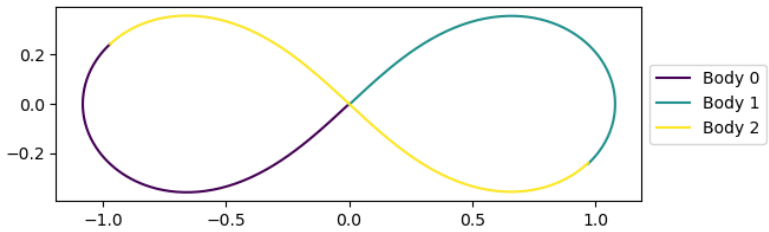


Figure 20: Integration of initial conditions with symmetry conditions imposed on the figure-8

With this in mind, restricting the Fourier coefficients of the sampled figure-8 and taking the initial conditions, it does visibly integrate to the figure-8 over a single period in Figure 20, albeit with a significant momentum error seen in Figure 21. Whilst the Fourier restrictions may spatial encode the correct symmetries, our orbit seems to lose to much information in the restriction and becomes divergent.

It is at this point that it is useful to highlight the importance of initializing the time variable of our trajectories properly for the FFT. These symmetry restrictions are intended for a specific configuration where $t = 0 \implies (x, y, z) = (0, 0, 0)$. Performing these symmetry restrictions on initial conditions for the figure-8 which do not satisfy the above property e.g.

$$\begin{aligned} x_1 &= (0.97000436, -0.24308753), & v_1 &= (0.4662036850, 0.4323657300), \\ x_2 &= (0.97000436, -0.24308753), & v_2 &= (0.4662036850, 0.4323657300), \\ x_3 &= (0.00000000, 0.00000000), & v_3 &= (-0.93240737, -0.86473146). \end{aligned}$$

gives Figure 22.

We see that the effect of these constraints causes the trajectory to significantly shrink and the direction of motion to reverse. The spatial figure-8 symmetry is still enforced. The Fourier coefficients of the initial figure-8 do not follow the restrictions and so have large components that are zeroed by the restriction, resulting from a significant loss of information when zeroing out low-frequency modes in particular, causing the figure to shrink. This realization was important in my understanding of the minimizer, and will be a necessary observation when it comes to drawing initial-guess trajectories of forms other than the figure-8, as we must ensure to begin the sampling of the trajectory from the correct place on the curve.

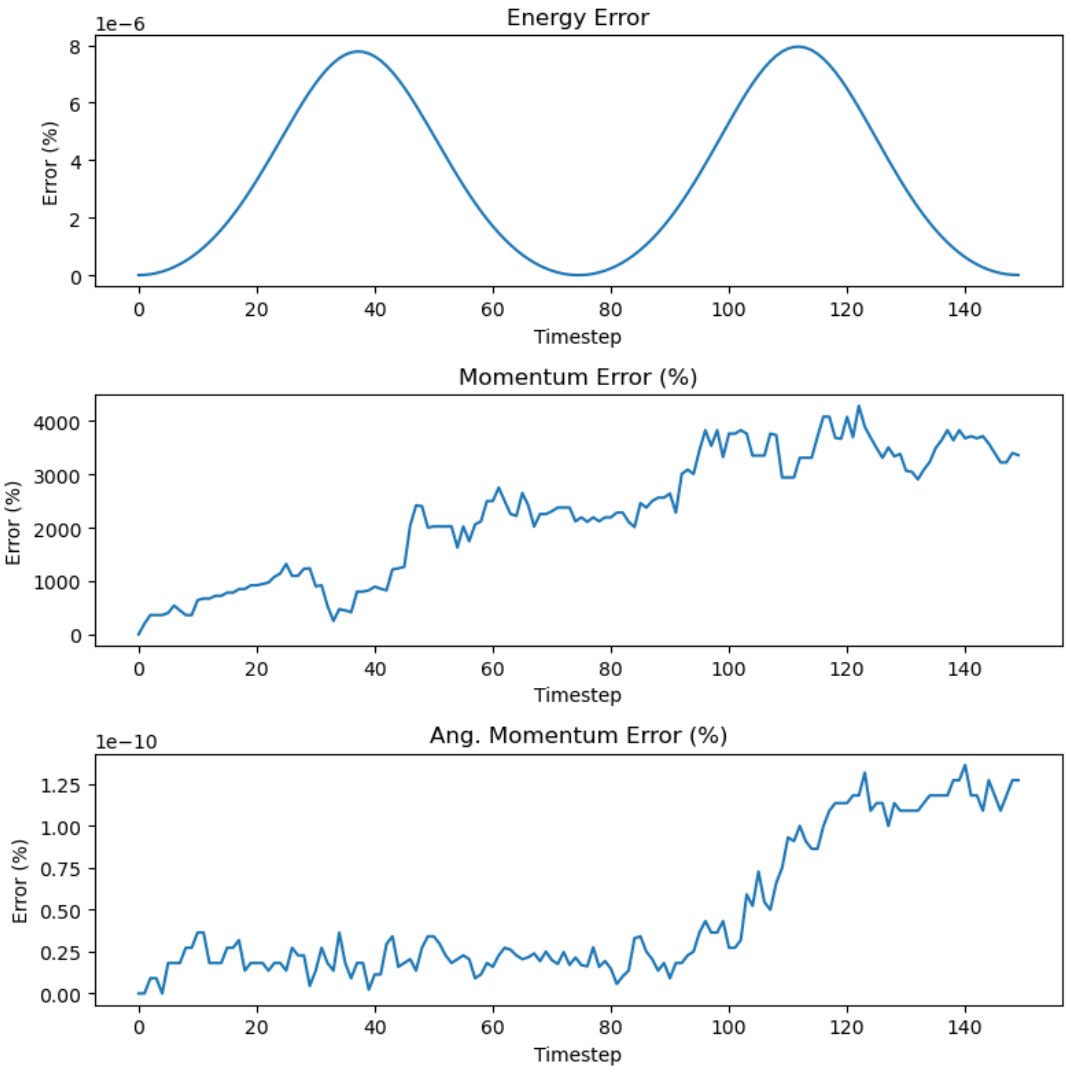
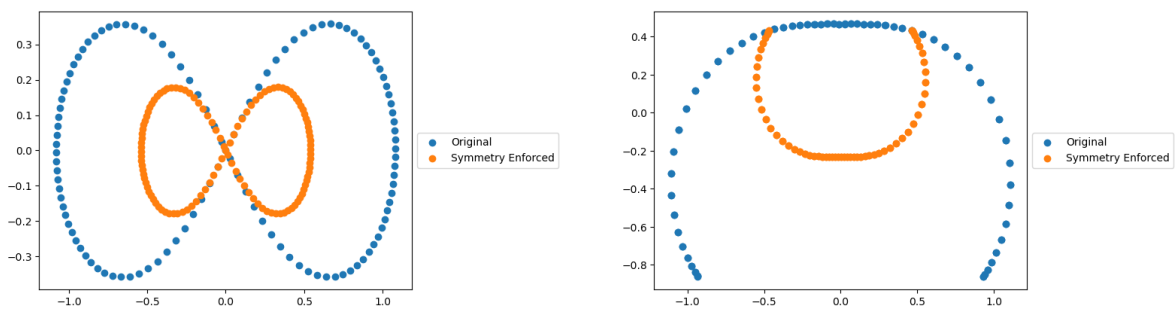


Figure 21: Errors of symmetry conditions imposed on figure-8 initial conditions



(a) Fourier restricted trajectory **(b)** Fourier restricted velocity

Figure 22: Incorrect figure-8 initial conditions, 132 samples, with D'(n, 2) restrictions on Fourier coefficients

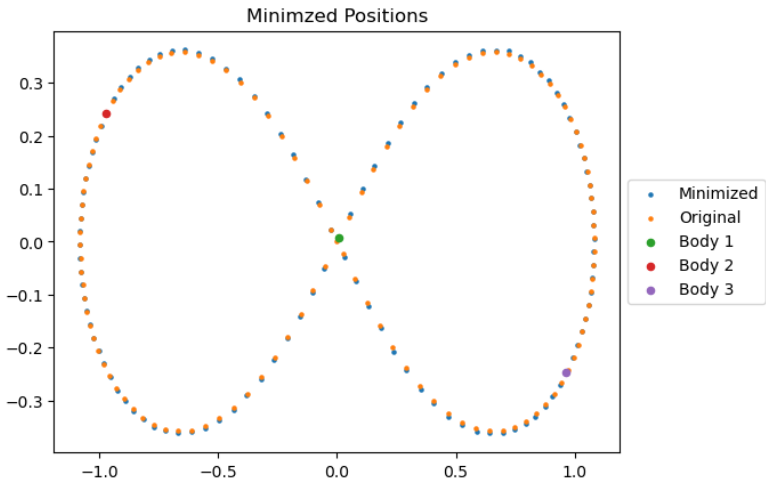


Figure 23: Minimized Trajectory

Now that we have a process to constrain the perturbation of our Fourier coefficients in the right directions for the spatial symmetries, we carry out the following process

- We feed the coefficients into *scipy.optimize.minimize* using the 'BFGS' method, seen in program *minimizer* (Appendix A 8). Our program takes the subset of unrestricted Fourier coefficients and perturbs them.
- It then unpacks these perturbed coefficients into the full set of Fourier coefficients with the symmetry restrictions now imposed, seen in program *Sx* (Appendix A 10).
- We then phase-shift these to get the coefficients for all other bodies in the system and apply the transformation to get the coefficients for the velocities of all bodies in the system.
- We can then transform these back to the configuration space, check that the result is entirely within the real plane (up to a small tolerance of imaginary parts due to accumulated errors) and use these positions and velocities to calculate the kinetic energy and potentials of the system, seen in programs *KE* (Appendix A 12) and *V* (Appendix A 13) respectively.
- We then measure the action along the loop using *scipy.integrate.simpson* and the numerical integrator, and the minimization program uses this to compare to the action of the original loop, seen in program *action* (Appendix A 14).

This process repeats until the program returns a convergence message with the respective Fourier coefficients, or a divergence message. If we have convergence, we transform these Fourier coefficients into the real space and run the checks described previously, and integrate the corresponding initial conditions to observe the choreography.

The first complete run of our program returns Figures 23 and 24 with a figure-8 input as the initial guess.

We see that the positions returned follow a figure-8 pattern closely, comparing the action of the initial guess and the minimized output we see . Looking at the velocities returned, we see a large discrepancy, this is also seen observing the Fourier coefficients. At this point it is not entirely apparent what the issue is. It could be due to the precision of the minimizer, not being able to

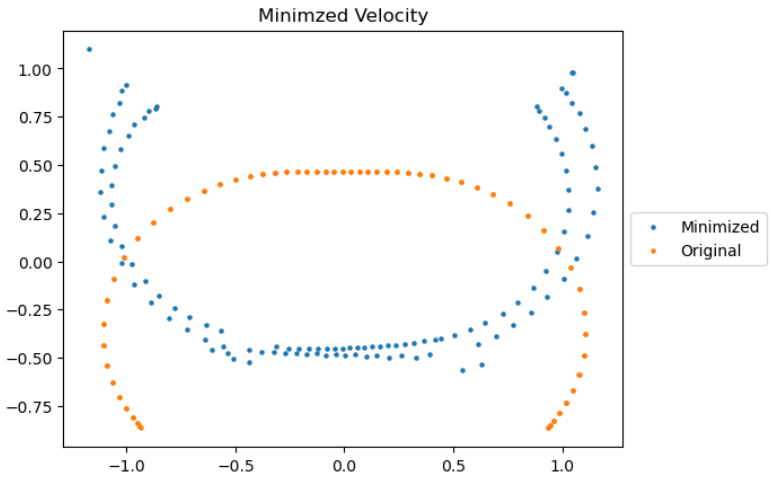
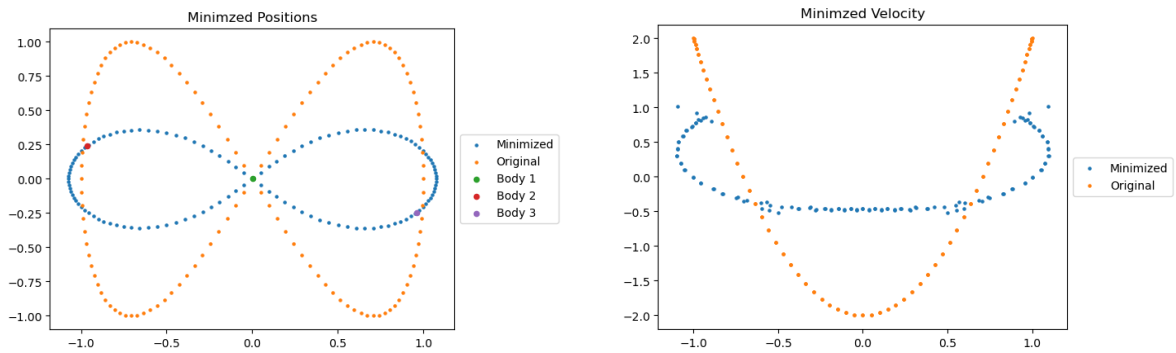


Figure 24: Minimized Velocity



(a) Original to Minimized trajectories

(b) Original to Minimized velocities

Figure 25: Lissajous 1:2 to figure-8

converge as close to the minimum as we like. The minimizer could not be working altogether. The restrictions on our Fourier coefficients could be entirely wrong or too strict, or there may be a bug in the code.

We first provide evidence that the minimizer is in fact working, and moving towards a minima. We minimize a Lissajous (1:2) figure as it is visibly different to the figure-8 allowing us to observe what the minimizer does more effectively. We restrict the Fourier coefficients and compare the action of the original and minimized curves.

It must first be noted that the minimizer returns a divergence message. The results we see are the minimal-action curve found by the program, but not necessarily the minima of the functional space. This will be addressed shortly.

The action of the original curve is given as $S[\mathbf{x}] = 35.177$, and the minimized curve $\hat{S}[\mathbf{x}] = 24.177$, confirming that the minimizer is in fact working here. We also see the resulting curve in Figure 25 much more closely resembles a figure-8 orbit than the initial curve. This suggests that whilst our Fourier constraints may not be entirely correct to give a physical figure-8 orbit, they are certainly close to the requirement as the minimal-action curve found in the functional space resembles the figure-8. The velocity curve remains noisier than expected. Integrating the resulting initial conditions leads to the trajectory seen in Figure 26.

We see that whilst divergent, the figure somewhat resembles a figure-8 initially. This suggests

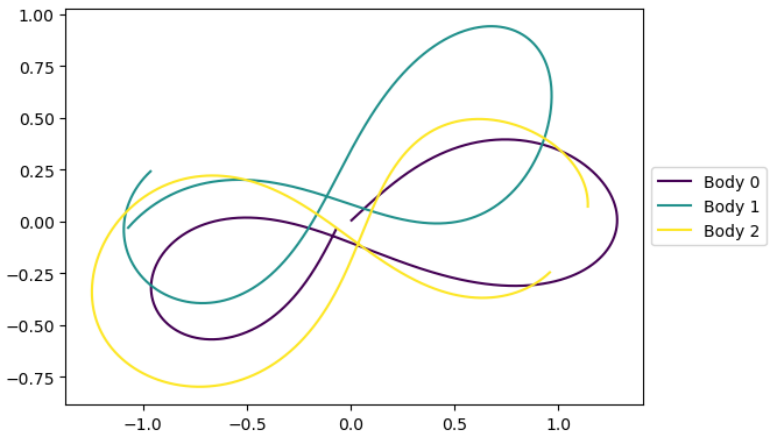


Figure 26: Minimized Lissajous 1:2 figure

the initial conditions are reasonable close to the figure-8 initial conditions. We see very similar results when minimizing figure-8 initial conditions.

Considering our restrictions on the Fourier coefficients, we saw in Figures 20 and 21 that the restriction alone on the figure-8 curve, without performing any minimization, integrated to a curve with large momentum error. This is not a physical curve, as momentum is a conserved quantity. This would therefore explain the divergence of our minimizer. The figure-8 orbit is a known choreography, and so is a minima in the $D'(n, 2)$ functional space. The divergence of this orbit under our Fourier restrictions means that this orbit does not exist in the functional space given by our Fourier restrictions, and could suggest that we are asking our minimizer to work on a space which in fact has no minima.

Changing the minimization routines in *scipy.optimize.minimize* to ones that are more accurate near minima, or adding an additional minimization after with another method lead to no noticeable change in the results. We can conclude that our problem is likely with the restriction of the Fourier coefficients, specifically the restrictions on $x(t)$ and $y(t)$. Given further time with the project, my immediate action would be to reconsider these constraints.

6 Further Topics and Conclusion

Having shown our results and identified weaknesses in our process, we will now conclude this paper. Before doing so, it would be useful to identify areas of future study that would be most fruitful to consider following on from this project.

- First and foremost, I would conduct more personal investigation into the Fourier coefficient restrictions placed on each symmetric group. Considering this as the key problem area in developing the minimizer, successfully enforcing these conditions will make the minimizer effective.
- Moving forward from the minimizer, considering thorough ways to provide "initial-guess" trajectories to feed into the minimizer looked to be a problem tackled in a wide range of manners. Possible methods were drawing and sampling curves by hand through a program, swarm algorithms or stochastic processes. The latter two options seem to be the most numerous and novel, and would provide near complete program automation.

- Further investigation into minimization methods would certainly increase the efficiency of the model. Significant problems are caused when minimizing to points in shallow basins, the use of a conjugate gradient minimizer would significantly speed up this process.

To conclude this project, we demonstrated the mathematical framework involved in finding choreographies of the n-body problem, and have successfully employed our own techniques to execute parts of these processes. Significant investigation was done into the suitability of particular numerical integrators, settling with a Yoshida method with variable timestep as the most suitable technique for considering the long-term stability of physical orbits and the Yoshida method with fixed timestep as the most suitable method for integration over short time scales. We clearly documented the mathematical approach to developing the minimizer, and highlighted the challenges and considerations necessary to consider when approaching this problem.

Given more time to locate the issues with our restrictions on Fourier coefficients, once implementing a method to guess initial choreographies, it is likely that classical result from the n-body problem could be replicated quite quickly. Whilst unsuccessful in replicating these results by the conclusion of this project, I hope that I have accurately documented the process in an accessible manner to those interested in pursuing this problem, as well as documenting the key time-consuming challenges and the intuition behind the ways in which I fixed these issues.

A Source Code

A.1 Integration

Listing 1: Python script for Euler-Cromer integration

```python
def nBodyEulerCromer(M, x0, v0, time):

    #Constants
    G = 1
    N = np.size(M)
    colors = plt.cm.viridis(np.linspace(0, 1, N))
    timesteps = np.shape(time)[0]
    dt = time[1] - time[0]

    #Centre of Mass
    CoM0 = np.zeros(3)
    v_CoM0 = np.zeros(3)
    total_mass = np.sum(M, axis= 0)

    for i in range(N):
        CoM0 += M[i] * x0[i, :]/total_mass
        v_CoM0 += M[i] * v0[i, :]/total_mass

    #Position/Velocity Vectors
    x = np.zeros((N, 3, timesteps))
    v = np.zeros((N, 3, timesteps))

    x[:, :, 0] = x0-CoM0
    v[:, :, 0] = v0-v_CoM0

    #Conserved Quantities
```

```
    T = np.zeros(timesteps) # KE
    U = np.zeros(timesteps) # PE
    P = np.zeros((3, timesteps)) # Momentum
    L = np.zeros((3, timesteps))

    for i in range(N):
        x_i = x[i, :, 0]
        u_i = v[i, :, 0]
        T_i = 0.5*M[i]*np.linalg.norm(u_i)**2
        P_i = M[i] * u_i
        L_i = np.cross(x_i, P_i)

        T[0] += T_i
        P[:, 0] += P_i
        L[:, 0] += L_i

        for j in range(i+1, N):
            r_ij = x[j, :, 0] - x[i, :, 0]
            U_ij = G*M[i]*M[j]/(np.linalg.norm(r_ij))
            U[0] -= U_ij

    for k in range(timesteps-1):
        for i in range(N):
            a_i = np.zeros(3)
            for j in range(N):
                x_i, x_j = x[i, :, k], x[j, :, k]
                r_ij = x_j - x_i
                if i > j:
                    #Calculate force exerted on i by each j
                    a_ij = (G*M[j]/(np.linalg.norm(r_ij)**3)) * (r_ij)
                    a_i += a_ij

                    #Calculating potentials
                    U_ij = G*M[i]*M[j]/(np.linalg.norm(r_ij))
                    U[k+1] -= U_ij
                elif i < j:
                    #Calculate force exerted on i by each j
                    a_ij = (G*M[j]/(np.linalg.norm(r_ij)**3)) * (r_ij)
                    a_i += a_ij

            #Euler Updates
            x_i, u_i = x[i, :, k], v[i, :, k]
            v[i, :, k+1] = (u_i + dt*a_i)

            v_i = v[i, :, k+1]
            x[i, :, k+1] = (x_i + dt*v_i)

            #Take KE of each particle at time t
            T_i = 0.5*M[i]*np.linalg.norm(v_i)**2
            T[k+1] += T_i

            #Take momentum of each particle at time k
            P_i = M[i]*v_i
            L_i = np.cross(x[i, :, k+1], P_i)

            P[:, k+1] += P_i
            L[:, k+1] += L_i

```

```python
    #Conserved Quantities Errors
    E = T + U
    P0, P_error_denominator = P[:, 0], P[:, 0]
    L0, L_error_denominator = L[:, 0], L[:, 0]
    E_error = np.abs((E-E[0])/E[0])

    if np.abs(E[0]) == 0:
        for i in range(N):
            v0_i = v0[i, :]
            if np.linalg.norm(v0_i) != 0:
                P_error_denominator = M[i]*v0_i
                break

    if np.linalg.norm(P[:, 0]) == 0:
        for i in range(N):
            v0_i = v0[i, :]
            if np.linalg.norm(v0_i) != 0:
                P_error_denominator = M[i]*v0_i
                break

    if np.linalg.norm(L[:, 0]) == 0:
        for i in range(N):
            x0_i = x0[i, :]
            v0_i = v0[i, :]
            P0_i = M[i]*v0_i
            L0_i = np.cross(x0_i, P0_i)
            if np.linalg.norm(L0_i) != 0:
                L_error_denominator = L0_i
                break

    P_error = np.linalg.norm((P-P0[:, None]), axis = 0)/np.linalg.norm(P_error_denominator)
    L_error = np.linalg.norm((L-L0[:, None]), axis = 0)/np.linalg.norm(L_error_denominator)

    # Plotting the orbits
    fig1, axa = plt.subplots()
    #axa.scatter(0, 0, color = 'black', label = "Centre of Mass")
    for i in range(N):
        axa.plot(x[i, 0, :], x[i, 1, :], color = colors[i], label = f"Body {i+1}")
        axa.scatter(x0[i, 0], x0[i, 1], color = colors[i], label = f"Mass {M[i]} Start ")
        axa.scatter(x[i, 0, -1], x[i, 1, -1], facecolor = 'None', edgecolor = colors[i], label = f"Mass {M[i]} End ")
        plt.gca().set_aspect('equal', adjustable='box')
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))

    fig2, axb = plt.subplots(3, 1, figsize = (10, 15))
    axb[0].plot(time, E_error)
    axb[0].set_title("Energy Error")
    axb[1].plot(time, P_error)
    axb[1].set_title("Momentum Error")
    axb[2].plot(time, L_error)
    axb[2].set_title("Ang. Momentum Error")
    plt.tight_layout()
    return x, v
```

Listing 2: Python script for Leapfrog integration

```python
def nBodyLeapfrog(M, x0, v0, time):

    #Constants
    G = 1
    N = np.size(M)
    colors = plt.cm.viridis(np.linspace(0, 1, N))
    timesteps = np.shape(time)[0]
    dt = time[1] - time[0]

    #Centre of Mass
    CoM0 = np.zeros(3)
    v_CoM0 = np.zeros(3)
    total_mass = np.sum(M, axis= 0)

    for i in range(N):
        CoM0 += M[i] * x0[i, :]/total_mass
        v_CoM0 += M[i] * v0[i, :]/total_mass

    #Position/Velocity Vectors
    x = np.zeros((N, 3, timesteps))
    v = np.zeros((N, 3, timesteps))
    v_plushalf = np.zeros((N, 3, timesteps))

    x[:, :, 0] = x0-CoM0
    v[:, :, 0] = v0-v_CoM0

    _, U0 = acceleration(x[:, :, 0], M, G)

    #Conserved Quantities
    T = np.zeros(timesteps) # KE
    U = np.zeros(timesteps) # PE
    P = np.zeros((3, timesteps)) # Momentum
    L = np.zeros((3, timesteps)) # Ang. Momentum

    T[0] = np.sum(0.5*M*np.linalg.norm(v[:, :, 0], axis = 1)**2)
    U[0] = U0

    for i in range(N):
        x_i = x[i, :, 0]
        u_i = v[i, :, 0]
        P_i = M[i] * u_i
        L_i = np.cross(x_i, P_i)

        P[:, 0] += P_i
        L[:, 0] += L_i

    for k in range(timesteps-1):
        a_k, U_k = acceleration(x[:, :, k], M, G)
        v_plushalf[:, :, k] = v[:, :, k] + 0.5*a_k*dt

        x[:, :, k+1] = x[:, :, k] + v_plushalf[:, :, k]*dt

        a_kp1, U_kp1 = acceleration(x[:, :, k+1], M, G)
        v[:, :, k+1] = v_plushalf[:, :, k] + 0.5*a_kp1*dt

        T[k+1] = np.sum(0.5*M@np.linalg.norm(v[:, :, k+1], axis = 1)
            **2)
```

```
        U[k+1] = U_kp1

        for i in range(N):
            x_i = x[i, :, k+1]
            v_i = v[i, :, k+1]
            P_i = M[i] * v_i
            L_i = np.cross(x_i, P_i)

            P[:, k+1] += P_i
            L[:, k+1] += L_i
    #Conserved Quantities Errors
    E = T + U
    P0, P_error_denominator = P[:, 0], P[:, 0]
    L0, L_error_denominator = L[:, 0], L[:, 0]
    E_error = np.abs((E-E[0])/E[0])

    if np.linalg.norm(P[:, 0]) == 0:
        print("Triggered")
        for i in range(N):
            v0_i = v0[i, :]
            if np.linalg.norm(v0_i) != 0:
                P_error_denominator = M[i]*v0_i
                break

    if np.linalg.norm(L[:, 0]) == 0:
        for i in range(N):
            x0_i = x0[i, :]
            v0_i = v0[i, :]
            P0_i = M[i]*v0_i
            L0_i = np.cross(x0_i, P0_i)
            if np.linalg.norm(L0_i) != 0:
                L_error_denominator = L0_i
                break

    P_error = np.linalg.norm((P-P0[:, None]), axis = 0)/np.linalg.norm(P_error_denominator)
    L_error = np.linalg.norm((L-L0[:, None]), axis = 0)/np.linalg.norm(L_error_denominator)

    # Plotting the orbits
    fig1, axa = plt.subplots()
    #axa.scatter(CoM0[0], CoM0[1], color = 'black', label = "Centre of Mass")
    for i in range(N):
        axa.plot(x[i, 0, :], x[i, 1, :], linestyle = 'None', color = colors[i], label = f"Body {i}", marker = 'o', markersize = 1)
        axa.scatter(x[i, 0, 0], x[i, 1, 0], color = colors[i], label = f"Mass {i} Start ")
        axa.scatter(x[i, 0, -1], x[i, 1, -1], facecolor = 'None', edgecolor = colors[i], label = f"Mass {i} End ")
        plt.gca().set_aspect('equal', adjustable='box')
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))

    fig2, axb = plt.subplots(3, 1, figsize = (10, 15))
    axb[0].plot(time, E_error)
    axb[0].set_title("Energy Error")
    axb[1].plot(time, P_error)
```

```
    axb[1].set_title("Momentum Error")
    axb[2].plot(time, L_error)
    axb[2].set_title("Ang. Momentum Error")
    plt.tight_layout()
    return x, v
```

Listing 3: Python script for Yoshida integration

```
def nBodyYoshida(M, x0, v0, dt, timesteps):

    #Constants
    G = 1
    N = np.size(M)
    colors = plt.cm.viridis(np.linspace(0, 1, N))
    time = np.arange(timesteps)

    #Centre of Mass
    CoM0 = np.zeros(3)
    v_CoM0 = np.zeros(3)
    total_mass = np.sum(M, axis= 0)

    for i in range(N):
        CoM0 += M[i] * x0[i, :]/total_mass
        v_CoM0 += M[i] * v0[i, :]/total_mass

    #Position/Velocity Vectors
    x = np.zeros((N, 3, timesteps))
    v = np.zeros((N, 3, timesteps))

    x[:, :, 0] = x0-CoM0
    v[:, :, 0] = v0-v_CoM0

    _, U0 = acceleration(x[:, :, 0], M, G)

    #Yoshida Coefficients
    w0 = -((2)**(1/3))/(2-2**(1/3))
    w1 = 1/(2-2**(1/3))
    c1 = w1/2
    c2 = (w0+w1)/2
    c3 = (w0+w1)/2
    c4 = w1/2
    d1 = w1
    d2 = w0
    d3 = w1

    #Conserved Quantities
    T = np.zeros(timesteps) # KE
    U = np.zeros(timesteps) # PE
    P = np.zeros((3, timesteps)) # Momentum
    L = np.zeros((3, timesteps)) #Angular Momentum

    T[0] = np.sum(0.5*M*np.linalg.norm(v[:, :, 0], axis = 1)**2)
    U[0] = U0

    for i in range(N):
        x_i = x[i, :, 0]
        u_i = v[i, :, 0]
        P_i = M[i] * u_i
```

```
        L_i = np.cross(x_i, P_i)

        P[:, 0] += P_i
        L[:, 0] += L_i

    for k in range(timesteps-1):

        #First Step
        x_int1 = x[:, :, k] + c1*v[:, :, k]*dt
        a_int1, _ = acceleration(x_int1, M, G)
        v_int1 = v[:, :, k] + d1*a_int1*dt

        #Second Step
        x_int2 = x_int1 + c2*v_int1*dt
        a_int2, _ = acceleration(x_int2, M, G)
        v_int2 = v_int1 + d2*a_int2*dt

        #Third Step
        x_int3 = x_int2 + c3*v_int2*dt
        a_int3, _ = acceleration(x_int3, M, G)
        v_int3 = v_int2 + d3*a_int3*dt

        #Final Step
        x[:, :, k+1] = x_int3 + c4*v_int3*dt
        _, U_int4 = acceleration(x[:, :, k+1], M, G)
        v[:, :, k+1] = v_int3


        T[k+1] = np.sum(0.5*M*(np.linalg.norm(v[:, :, k+1], axis = 1)
            **2))
        U[k+1] = U_int4

        for i in range(N):
            x_i = x[i, :, k+1]
            v_i = v[i, :, k+1]
            P_i = M[i] * v_i
            L_i = np.cross(x_i, P_i)

            P[:, k+1] += P_i
            L[:, k+1] += L_i
    #Conserved Quantities Errors
    E = T + U
    P0, P_error_denominator = P[:, 0], P[:, 0]
    L0, L_error_denominator = L[:, 0], L[:, 0]
    E_error = np.abs((E-E[0])/E[0])

    if np.linalg.norm(P[:, 0]) == 0:
        for i in range(N):
            v0_i = v0[i, :]
            if np.linalg.norm(v0_i) != 0:
                P_error_denominator = M[i]*v0_i
                break

    if np.linalg.norm(L[:, 0]) == 0:
        for i in range(N):
            x0_i = x0[i, :]
            v0_i = v0[i, :]
            P0_i = M[i]*v0_i
```

```
            L0_i = np.cross(x0_i, P0_i)
            if np.linalg.norm(L0_i) != 0:
                L_error_denominator = L0_i
                break

    P_error = np.linalg.norm((P-P0[:, None]), axis = 0)/np.linalg.norm(P_error_denominator)
    L_error = np.linalg.norm((L-L0[:, None]), axis = 0)/np.linalg.norm(L_error_denominator)

    # Plotting the orbits
    fig1, axa = plt.subplots()
    #axa.scatter(0, 0, color = 'black', label = "Centre of Mass")
    for i in range(N):
        axa.plot(x[i, 0, :], x[i, 1, :], color = colors[i], label = f"Body {i}")
        axa.scatter(x[i, 0, 0], x[i, 1, 0], color = colors[i], label = f"Mass {i} Start ")
        axa.scatter(x[i, 0, -1], x[i, 1, -1], facecolor = 'None', edgecolor = colors[i], label = f"Mass {i} End ")
        plt.gca().set_aspect('equal', adjustable='box')
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))

    fig2, axb = plt.subplots(3, 1, figsize = (8, 8))
    axb[0].plot(time, 100*E_error)
    axb[0].set_title("Energy Error")
    axb[0].set_ylabel("Error (%)")
    axb[0].set_xlabel("Timestep")
    axb[1].plot(time, 100*P_error)
    axb[1].set_title("Momentum Error (%)")
    axb[1].set_ylabel("Error (%)")
    axb[1].set_xlabel("Timestep")
    axb[2].plot(time, 100*L_error)
    axb[2].set_title("Ang. Momentum Error (%)")
    axb[2].set_ylabel("Error (%)")
    axb[2].set_xlabel("Timestep")
    plt.tight_layout()
    return x, v, U, T
```

Listing 4: Python script for Yoshida integration with variable timestep

```
def nBodyYoshidaVT_Plots(M, G, x0, v0, timesteps, eta):
    #Initialise
    N = np.size(M)
    colors = plt.cm.viridis(np.linspace(0, 1, N))
    time = np.zeros(timesteps)
    t_collapse = timesteps

    #Yoshida Coefficients
    w0 = -((2)**(1/3))/(2-2**(1/3))
    w1 = 1/(2-2**(1/3))
    c1 = w1/2
    c2 = (w0+w1)/2
    c3 = (w0+w1)/2
    c4 = w1/2
    d1 = w1
    d2 = w0
    d3 = w1
```

```python

    constants = [M, G, c1, c2, c3, c4, d1, d2, d3]

    #Centre of Mass
    CoM0 = np.zeros(3)
    v_CoM0 = np.zeros(3)
    total_mass = np.sum(M, axis= 0)

    for i in range(N):
        CoM0 += M[i] * x0[i, :]/total_mass
        v_CoM0 += M[i] * v0[i, :]/total_mass

    _, U0, _ = acceleration(x0, M, G)

    #Position/Velocity Vectors
    x = np.zeros((N, 3, timesteps))
    v = np.zeros((N, 3, timesteps))

    x[:, :, 0] = x0-CoM0
    v[:, :, 0] = v0-v_CoM0

    #Conserved Quantities
    E = np.zeros(timesteps)
    T = np.zeros(timesteps) # KE
    U = np.zeros(timesteps) # PE
    P = np.zeros((3, timesteps)) # Momentum
    L = np.zeros((3, timesteps)) #Angular Momentum

    T[0] = np.sum(0.5*M*np.linalg.norm(v[:, :, 0], axis = 1)**2)
    U[0] = U0
    E[0] = T[0] + U[0]
    P[:, 0] = np.sum(M[:, None] * v[:, :, 0], axis=0)
    L[:, 0] = np.sum(np.cross(x[:, :, 0], M[:, None] * v[:, :, 0]),
        axis=0)

    #Errors
    E_error = np.zeros(timesteps)
    P_error = np.zeros(timesteps)
    L_error = np.zeros(timesteps)

    E_error_denominator = np.absolute(E[0])
    P_error_denominator = np.linalg.norm(P[:, 0])
    L_error_denominator = np.linalg.norm(L[:, 0])

    #Check for zeros
    if np.linalg.norm(P[:, 0]) == 0:
        for i in range(N):
            v0_i = v0[i, :]
            if np.linalg.norm(v0_i) != 0:
                P_error_denominator = np.linalg.norm(M[i]*v0_i)
                break

    if np.linalg.norm(L[:, 0]) == 0:
        for i in range(N):
            x0_i = x0[i, :]
            v0_i = v0[i, :]
            P0_i = M[i]*v0_i
            L0_i = np.cross(x0_i, P0_i)
```

```
            if np.linalg.norm(L0_i) != 0:
                L_error_denominator = np.linalg.norm(L0_i)
                break

    for k in range(timesteps-1):
        #Finding dt
        dt_forward = timestep(x[:, :, k], v[:, :, k], eta, N)
        x_forward, v_forward, _, _ = KickDriftKick(x[:, :, k], v[:, :, k], dt_forward, constants)
        dt_back = timestep(x_forward, v_forward, eta, N)
        dt = (dt_forward+dt_back)/2
        time[k+1] = time[k] + dt

        #Using dt to find x, v
        x[:, :, k+1], v[:, :, k+1], U[k+1], collapse= KickDriftKick(x[:, :, k], v[:, :, k], dt, constants)

        #Update Conserved Quantities
        T[k+1] = np.sum(0.5*M*(np.linalg.norm(v[:, :, k+1], axis = 1)**2))
        E[k+1] = T[k+1] + U[k+1]
        P[:, k+1] = np.sum(M[:, None] * v[:, :, k+1], axis=0)
        L[:, k+1] = np.sum(np.cross(x[:, :, k+1], M[:, None] * v[:, :, k+1]), axis=0)

        #Check Errors
        E_error[k+1] = np.absolute((E[k+1]-E[0]))/E_error_denominator
        P_error[k+1] = np.linalg.norm(P[:, k+1] - P[:, 0])/P_error_denominator
        L_error[k+1] = np.linalg.norm(L[:, k+1] - L[:, 0])/L_error_denominator

        if P_error[k+1] > 1e-10:
            return f"Linear Momentum Error Too High, {P_error[k+1]*100}%"
        if L_error[k+1] > 1e-10:
            return f"Angular Momentum Error Too High, {L_error[k+1]*100}%"
        if E_error[k+1] > 1e-3:
            return f"Energy Error Too High, {E_error[k+1]*100}%"

        #Check for system collapse
        if collapse == True:
            print("System has collapsed")
            t_collapse = k
            break

    # Plotting the orbits
    fig1, axa = plt.subplots()
    #axa.scatter(0, 0, color = 'black', label = "Centre of Mass")
    for i in range(N):
        axa.plot(x[i, 0, 0:t_collapse], x[i, 1, 0:t_collapse], color = colors[i], label = f"Body {i+1}")
        axa.scatter(x[i, 0, 0], x[i, 1, 0], color = colors[i], label = f"Mass {i} Start ")
        axa.scatter(x[i, 0, -1], x[i, 1, -1], facecolor = 'None', edgecolor = colors[i], label = f"Mass {i} End ")
        plt.gca().set_aspect('equal', adjustable='box')
```

```
    plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))

    time = time[0:t_collapse]
    fig2, axb = plt.subplots(3, 1, figsize = (8, 8))
    axb[0].plot(time, 100*E_error)
    axb[0].set_title("Energy Error")
    axb[0].set_ylabel("Error (%)")
    axb[0].set_xlabel("Time")
    axb[1].plot(time, 100*P_error)
    axb[1].set_title("Momentum Error")
    axb[1].set_ylabel("Error (%)")
    axb[1].set_xlabel("Time")
    axb[2].plot(time, 100*L_error)
    axb[2].set_title("Ang. Momentum Error")
    axb[2].set_ylabel("Error (%)")
    axb[2].set_xlabel("Time")
    plt.tight_layout()
    return x, v
```

Listing 5: Python script for Acceleration function

```
def acceleration(x, M, G):
    N = np.shape(x)[0]
    a = np.zeros((N, 3))
    U = 0
    collapse = False

    for i in range(N):
        for j in range(i+1, N):
            x_i, x_j = x[i, :], x[j, :]
            r_ij = x_j - x_i
            r_ij_norm = np.linalg.norm(r_ij) + 1e-7

            if r_ij_norm < 1e-7 or r_ij_norm > 20:
                collapse = True

            #Calculate force exerted on i by each j
            a_ij = G*M[i]*M[j]/(r_ij_norm**3) * (r_ij)
            a[i, :] += a_ij/M[i]
            a[j, :] -= a_ij/M[j]

            #Calculating potentials
            U_ij = -G*M[i]*M[j]/(r_ij_norm)
            U += U_ij

    return a, U, collapse
```

Listing 6: Python script for Variable Timestep function

```
def timestep(x, v, eta, N): #Max dt = 1
    dt = 1
    for i in range(N):
        for j in range(i+1, N):
            x_i, x_j = x[i, :], x[j, :]
            r_ij = x_j - x_i
            v_i, v_j = v[i, :], v[j, :]
            v_ij = v_j - v_i
            dt_ij = np.linalg.norm(r_ij)/(np.linalg.norm(v_ij)+1e-7)
```

```
            dt = np.minimum(dt, dt_ij)
    return eta*dt
```

Listing 7: Python script for Kick-Drfit-Kick function

```
def KickDriftKick(x, v, dt, constants):
    M, G, c1, c2, c3, c4, d1, d2, d3 = constants
    #First Step
    x_int1 = x + c1*v*dt
    a_int1, _, _ = acceleration(x_int1, M, G)
    v_int1 = v + d1*a_int1*dt

    #Second Step
    x_int2 = x_int1 + c2*v_int1*dt
    a_int2, _, _ = acceleration(x_int2, M, G)
    v_int2 = v_int1 + d2*a_int2*dt

    #Third Step
    x_int3 = x_int2 + c3*v_int2*dt
    a_int3, _, _ = acceleration(x_int3, M, G)
    v_int3 = v_int2 + d3*a_int3*dt

    #Final Step
    x_kp1 = x_int3 + c4*v_int3*dt
    _, U_int4, collapse = acceleration(x_kp1, M, G)
    v_kp1 = v_int3
    return x_kp1, v_kp1, U_int4, collapse
```

A.2 Minimization

Listing 8: Python script for Minimization function

```
def minimizer(samples, N, T):
    #Find frequencies, angular velocity, and coefficients
    K = np.shape(samples)[1]
    omega = 2*np.pi*np.fft.fftfreq(K, d = T/K)
    coefficients = np.fft.rfft(samples, axis = 1)

    initial_coefficients, indices = Symmetry_D2dash(coefficients, N, K,
        None, "Enforce␣Restriction")

    #Minimize
    min_fourier_coefficients_int = minimize(Sx, initial_coefficients,
        method = 'L-BFGS-B', args=(N, K, indices, T))
    min_fourier_coefficients = minimize(Sx,
        min_fourier_coefficients_int.x, method='Nelder-Mead', options={'
        xatol': 1e-12, 'fatol': 1e-12, 'maxiter': 2000}, args=(N, K,
        indices, T))

    #Unpack coefficients
    x_fourier_coefficients, KUr = Symmetry_D2dash(
        min_fourier_coefficients.x, N, K, indices, "Undo␣Restriction")

    #Generate velocity coefficients
    v_fourier_coefficients = 1j*omega[None, :]*x_fourier_coefficients

```

```
    #Move to real space
    x = np.fft.ifft(x_fourier_coefficients, axis = 1).real
    v = np.fft.ifft(v_fourier_coefficients, axis = 1).real

    imaginary_parts_x, imaginary_parts_v = check_imaginary_parts(x), check_imaginary_parts(v)

    return [x, v], [x_fourier_coefficients, v_fourier_coefficients], [min_fourier_coefficients.success, imaginary_parts_x, imaginary_parts_v]
```

Listing 9: Python script for Symmetry function

```
def Symmetry_D2dash(coefficients, N, K, indices, direction):
    '''Takes a set of coefficients and applies/removes the symmetry groups restrictions
    When applying them, using direction == "Enforce Restriction", a 1 dim array is returned of all real parts
    of coefficients to be perturbed, then all imaginary parts. It also returns the number of restricted coefficients
    When removing them, using direction == "Undo Restriction", it first removes the CoM restrictions, then reconstructs a set
    of K (= no of samples) coefficients
    '''
    if direction == "Enforce Restriction":
        K = np.shape(coefficients)[1]

        #CoM Restriction
        indices = np.arange(K)
        CoM_mask = (indices % N != 0)

        # Even and Odd masks (excluding CoM restriction)
        Even_mask = (indices % 2 == 0) & CoM_mask
        Odd_mask  = (indices % 2 == 1) & CoM_mask

        #Restrict
        x_real = coefficients[0, :][Even_mask].real
        x_imag = coefficients[0, :][Odd_mask].imag
        y_real = coefficients[1, :][Odd_mask].real
        y_imag = coefficients[1, :][Even_mask].imag

        #Return restricted coefficients
        restricted_coefficients = np.concatenate((x_real, x_imag, y_real, y_imag))
        return restricted_coefficients, [Even_mask, Odd_mask, CoM_mask],

    elif direction == "Undo Restriction":
        #Unpack index
        [Even_mask, Odd_mask, CoM_mask] = indices
        K_even = (Even_mask & CoM_mask).sum()
        K_odd = (Odd_mask & CoM_mask).sum()

        #Unpack coefficients
        x_restricted_real = coefficients[:K_even]
        x_restricted_imag = coefficients[K_even:(K_even+K_odd)]
        y_restricted_real = coefficients[(K_even+K_odd):(K_even+2*K_odd)]
```

```python
        y_restricted_imag = coefficients[(K_even+2*K_odd):]

        fourier_coefficients = np.zeros((2, K), dtype = complex)
        for i in range(1, K//2):
            if Even_mask[i] == True:
                fourier_coefficients[0, i] = x_restricted_real[0]
                fourier_coefficients[0, K - i] = x_restricted_real[0]
                x_restricted_real = x_restricted_real[1:]

                fourier_coefficients[1, i] = 1j*y_restricted_imag[0]
                fourier_coefficients[1, K - i] = -1j*y_restricted_imag[0]
                y_restricted_imag = y_restricted_imag[1:]

            elif Odd_mask[i] == True:
                fourier_coefficients[0, i] = 1j*x_restricted_imag[0]
                fourier_coefficients[0, K - i] = -1j*x_restricted_imag[0]
                x_restricted_imag = x_restricted_imag[1:]

                fourier_coefficients[1, i] = y_restricted_real[0]
                fourier_coefficients[1, K - i] = y_restricted_real[0]
                y_restricted_real = y_restricted_real[1:]

        kur = np.shape(fourier_coefficients)
        return fourier_coefficients, kur

    else:
        return "Invalid entry for direction parameter."
```

Listing 10: Python script for Action Functional

```python
def Sx(coeffs, N, K, indices, T):
    #Find frequencies and angular velocity
    omega = 2*np.pi*np.fft.fftfreq(K, d = T/K)

    #Unpack coefficients
    x_fourier_coefficients, KUr = Symmetry_D2dash(coeffs, N, K, indices, "Undo Restriction")

    #Generate velocity coefficients
    v_fourier_coefficients = 1j*omega[None, :]*x_fourier_coefficients

    #Calculate action
    KE = kinetic_energy(v_fourier_coefficients[0, :], v_fourier_coefficients[1, :], K, N, T)
    V = potential(x_fourier_coefficients[0, :], x_fourier_coefficients[1, :], K, N, T)
    Sx = action(KE, V, K, T)
    return Sx
```

Listing 11: Python script for function to check a trajectory is real

```python
def check_imaginary_parts(arr, tol=1e-14):
    imag_parts = np.abs(np.imag(arr))
    max_imag = imag_parts.max()
    has_large_imag = max_imag > tol
    return has_large_imag
```

Listing 12: Python script for kinetic energy calculation function

```python
def kinetic_energy(vx_fourier_coefficients, vy_fourier_coefficients, K, N, T):
    #Move to real space for all N bodies
    KE = np.zeros(K)
    omega = 2*np.pi*np.fft.fftfreq(K, d = T/K)
    for i in range(N):
        #Shift to find velocity coefficients of each body
        vx_i_fourier_coefficients = vx_fourier_coefficients * np.exp(-1j * omega * i * T / N)
        vy_i_fourier_coefficients = vy_fourier_coefficients * np.exp(-1j * omega * i * T / N)

        Vx_i = np.fft.ifft(vx_i_fourier_coefficients).real
        Vy_i = np.fft.ifft(vy_i_fourier_coefficients).real

        KE_i = 0.5*((Vx_i**2)+(Vy_i**2))
        KE += KE_i
    return KE
```

Listing 13: Python script for potential calculation function

```python
def potential(rx_fourier_coefficients, ry_fourier_coefficients, K, N, T):
    omega = 2*np.pi*np.fft.fftfreq(K, d = T/K)
    x = np.zeros((N, 2, K))
    for i in range(N):
        rx_i_fourier_coefficients = rx_fourier_coefficients * np.exp(-1j * omega * i * T / N)
        ry_i_fourier_coefficients = ry_fourier_coefficients * np.exp(-1j * omega * i * T / N)
        x[i, 0, :] = np.fft.ifft(rx_i_fourier_coefficients).real
        x[i, 1, :] = np.fft.ifft(ry_i_fourier_coefficients).real

    # Vectorized pairwise potential calculation
    V = np.zeros(K)

    for t in range(K):
        for i in range(N):
            for j in range(i+1, N):
                x_i, x_j = x[i, :, t], x[j, :, t]
                r_ij = x_j - x_i
                r_ij_norm = np.linalg.norm(r_ij)

                #Calculating potentials
                V[t] -= 1/(r_ij_norm + 1e-5)
    return V
```

Listing 14: Python script for action calculation function

```python
def action(KE, V, K, T):
    time = np.linspace(0, T, K, endpoint = False)
    L = KE - V
    Sx = simpson(L, x=time)
    return Sx
```

References

[1] Donald D Mueller Roger Redmond Bate and Jerry E White. *Fundamentals of Astrodynamics*. Courier Dover Publications, 1971.

[2] Joseph Louis Lagrange. 'Essai sur le Problème des Trois Corps'. In: *Oeuvres de Lagrange* (1772).

[3] Christopher Moore. 'Braids in Classical Dynamics'. In: *Physical Review Letters* (1993).

[4] Alain Chenciner and Richard Montgomery. 'A Remarkable Periodic Solution of the Three-Body Problem in the Case of Equal Masses'. In: *Annals of Mathematics Second Series* (2000).

[5] Carles Simó. 'New Families of Solutions in the n-Body Problem'. In: *European Congress of Mathematics* (2001).

[6] Gerhard Wanner Ernst Hairer and Christian Lubich. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. Springer, 2006.

[7] Haruo Yoshida. 'Symplectic Integrators for Hamiltonian Systems'. In: *IAU Symposia* (1992).

[8] James Montaldi and Katrina Steckles. 'Classifications of Symmetry Groups for n-Body Choreographies'. In: *Forum of Mathematics, Sigma* 1 (2013). ISSN: 2050-5094. DOI: 10.1017/fms.2013.5. URL: http://dx.doi.org/10.1017/fms.2013.5.

[9] Davide L ferrario and Susanna Terracini. 'On the Existence of Collisionless Equivariant Minimizers for the Classical n-Body Problem'. In: *Inventiones mathematicae* (2004).

[10] Hugh Brickman. 'Choreographies'. MA thesis. University of Edinburgh, School of Mathematics, 2014.

[11] Christopher D'Arcy. 'A Structured Approach to Finding n-Body Choreographies'. MA thesis. University of Edinburgh, School of Mathematics, 2015.